

Certification : Concepteur Développeur d'Applications (CDA)

EEG Explorer



Projet développé par **Marine PERNICI**

Février 2024

Spectre Biotech



&

Wild Code School



TABLE DES MATIERES

Table des Figures	4
I. Introduction	5
II. Liste des compétences du référentiel couverte par le projet	6
A. Concevoir et développer des composants d'interface utilisateur en intégrant les recommandations de sécurité	6
B. Concevoir et développer la persistance des données en intégrant les recommandations de sécurité	6
C. Concevoir et développer une application multicouche répartie en intégrant les recommandations de sécurité	6
D. Compétences transversales	6
III. Résumé du projet	7
IV. Cahier des charges	8
A. Contexte	8
B. Objectif.....	8
C. Fonctionnalités prioritaires	8
D. Périmètre	9
E. Ressources.....	9
F. Performance	9
V. Gestion de projet	10
A. Organisation et travail en équipe.....	10
B. Planification des activités	12
VI. Spécifications fonctionnelles	13
A. Attentes principales.....	13
B. Cas d'usage	13
C. Conception graphique et fonctionnelle	14
D. Arborescence fonctionnelle pour l'utilisateur	16
VII. Spécifications techniques	17
A. Environnement.....	17
B. Interface utilisateur.....	17
C. Backend et Base de Données.....	18
D. Authentification	18
E. Traitement du Langage Naturel & IA	18

F.	Architecture.....	19
G.	Sécurité et Conformité	19
H.	Déploiement.....	19
VIII.	Réalisations significatives	21
A.	Un front-end responsive et accessible	21
B.	Une interface interactive	25
C.	Les bases de données.....	27
1.	Base de données EEG Explorer.....	27
2.	Base de données Spectre	29
D.	Back-End : Exemple de l'agent SQL	30
E.	API.....	35
F.	Sécurité.....	37
1.	Cryptage des données sensibles.....	37
2.	Vérification des données entrantes	37
3.	Sécuriser l'accès à la base de données Spectre	38
4.	Sécuriser le transfert des données	39
5.	Isoler l'application avec docker.....	39
G.	Tests.....	40
1.	Exemple de tests unitaires	41
2.	Exemple de tests d'intégration	44
IX.	Jeu d'essai d'une fonctionnalité représentative du projet.....	47
X.	Veille sur les vulnérabilités de sécurité.....	50
XI.	Recherche de solution en Situation de travail	53
XII.	Conclusion	55
XIII.	Annexes.....	56
1.	Annexe 1 : Exemples d'éléments et de requêtes.....	56
2.	Annexe 2 : Déroulé temporel du projet EEG EXPLORER.....	57
3.	Annexe 3 : Premiers wireframes d'EEG Explorer	58
4.	Annexe 4 : Charte graphique d'EEG Explorer	59
5.	Annexe 5 : Maquettes de l'application EEG Explorer.....	61
6.	Annexe 6 : Fichier de configuration de l'agent SQL	62
7.	Annexe 7 : Exemple du fonctionnement de l'agent SQL.....	63

TABLE DES FIGURES

Figure 1 : Illustration du workflow Gitflow.....	11
Figure 2 : Capture d'écran du backlog sur Notion.....	12
Figure 3 : Diagramme des cas d'utilisation générale	13
Figure 4 : Exemples de wireframes	14
Figure 5 : Logo et palette de couleurs définis dans la charte graphique	15
Figure 6 : Exemples de maquettes	15
Figure 7 : Architecture fonctionnelle de l'application EEG Explorer.....	16
Figure 8 : Architecture multicouche et technologies principales d'EEG Explorer.....	19
Figure 9 : Rendu responsive de la page d'inscription et de connexion.....	21
Figure 10 : Rendu de la page d'accueil en cas de daltonisme	24
Figure 11 : Rendu visuel des feedbacks et du bouton d'inscription.....	26
Figure 12 : Diagramme de la base de données EEG Explorer.....	27
Figure 13 : Diagramme de la base de données Spectre	29
Figure 14 : Affichage du résultat de la requête	34
Figure 15 : Schéma des routes API de l'application EEG Explorer.	35
Figure 16 : Capture d'écran des résultats Pytest pour la sécurité du mot de passe	43
Figure 17 : Tableau des résultats du jeu d'essai.....	48
Figure 18 : Capture d'écran des résultats Pytest pour la fonction de routage	49
Figure 19 : Page d'accueil de la documentation de It's Dangerous.....	53

Nota Bene

Il y a deux bases de données distinctes dans ce projet :

- La **base de données Spectre** (qui sera mentionnée en vert) est consultée par l'outil d'intelligence artificielle pour répondre aux questions des utilisateurs.
- La **base de données EEG Explorer** (qui sera mentionnée en rouge) contient les informations générées par les utilisateurs de l'application.

I. INTRODUCTION

Après 13 ans d'expérience en tant que **neuropsychologue**, j'ai entrepris un tournant décisif en 2022 en me réorientant vers le domaine de l'informatique, avec un intérêt particulier pour **l'analyse et la science des données**. Ma formation de Data Analyst à la Wild Code School s'est avérée être une révélation, m'ouvrant les portes à un univers fascinant. Cette première étape m'a conduit à un **stage de 6 mois chez Spectre Biotech**, une startup dynamique spécialisée dans l'EEG, où j'ai pu allier ma passion pour les **neurosciences et l'informatique**.

Spectre Biotech, co-fondée par des experts en neuropsychologie, se dédie à **rendre l'imagerie cérébrale, notamment l'EEG (ElectroEncéphaloGraphie), plus accessible en pratique clinique**. Leur approche innovante combine un logiciel d'enregistrement EEG et une plateforme web pour une **analyse automatisée et accessible des données**. Ce stage fut une opportunité unique de conjuguer mes connaissances antérieures en neuropsychologie avec mes nouvelles compétences en data et il a été très **enrichissant**, tant sur le plan humain que professionnel.

Pour poursuivre mon apprentissage et renforcer mes compétences, j'ai ensuite entrepris un **contrat d'alternance en Data Science à la Wild Code School** tout en continuant mon parcours chez Spectre Biotech. Dans ce contexte de startup innovante, **la polyvalence est essentielle**, j'ai ainsi pu travailler sur des projets très divers, chacun présentant ses propres **défis et opportunités d'apprentissage**, de l'analyse et la visualisation de données complexes, à l'implémentation de tests unitaires, la gestion de projet, ou encore l'ajout de composants en front-end. Chaque projet m'a permis de me **familiariser avec de nouvelles technologies et méthodologies**, élargissant ainsi mon éventail de compétences techniques et professionnelles. Mon choix de reconversion a été conforté et j'ai **gagné en confiance** sur mes nouvelles compétences et ma capacité à encore en apprendre de nouvelles.

Dans ce cadre, le **projet EEG Explorer** représente une synthèse de mes connaissances en data science et en développement. C'est une application web conçue pour **simplifier l'accès aux données complexes d'EEG**. Grâce à une interface intuitive et à l'intégration d'un **modèle d'intelligence artificielle**, elle permet un accès simplifié et efficace à des informations cruciales stockées en base de données, ouvrant de nouvelles perspectives tant pour les professionnels de la santé que pour la recherche en neurosciences.

La réalisation de ce projet dans le cadre de la certification CDA a été **un véritable défi** qui m'a permis de mettre en œuvre les compétences acquises précédemment et de monter en compétences sur toutes celles dont j'ai eu besoin par ailleurs, en IA comme en développement applicatif web.

II. LISTE DES COMPETENCES DU REFERENTIEL COUVERTE PAR LE PROJET

Le projet EEG Explorer présenté dans ce document a pour ambition de **couvrir l'ensemble des compétences obligatoires** décrites dans le référentiel de certification du titre professionnel de Concepteur Développeur d'Applications (CDA) et listées ci-après.

A. CONCEVOIR ET DEVELOPPER DES COMPOSANTS D'INTERFACE UTILISATEUR EN INTEGRANT LES RECOMMANDATIONS DE SECURITE

- Maquetter une application
- Développer une interface utilisateur de type desktop
- Développer des composants d'accès aux données
- Développer la partie front-end d'une interface utilisateur web
- Développer la partie back-end d'une interface utilisateur web

B. CONCEVOIR ET DEVELOPPER LA PERSISTANCE DES DONNEES EN INTEGRANT LES RECOMMANDATIONS DE SECURITE

- Concevoir une base de données
- Mettre en place une base de données
- Développer des composants dans le langage d'une base de données

C. CONCEVOIR ET DEVELOPPER UNE APPLICATION MULTICOUCHE REPARTIE EN INTEGRANT LES RECOMMANDATIONS DE SECURITE

- Collaborer à la gestion d'un projet informatique et à l'organisation de l'environnement de développement
- Concevoir une application
- Développer des composants métier
- Construire une application organisée en couches
- Développer une application mobile
- Préparer et exécuter les plans de tests d'une application
- Préparer et exécuter le déploiement d'une application

D. COMPETENCES TRANSVERSALES

- Utiliser l'anglais dans son activité professionnelle en conception et développement d'applications
- Actualiser et partager ses compétences en conception et développement d'applications

III. RESUME DU PROJET

The EEG Explorer project was carried out as part of a **Data Scientist work-study program at Wild Code School and within Spectre Biotech**. Since 2021, this startup has been developing and marketing **a solution for the automated processing of electroencephalography (EEG) data**.

The aim was to design a secure application enabling Spectre Biotech's non-technical managers to **query their database in natural language**. This database contains clinical information on users, as well as the results of their EEG recordings.

EEG Explorer is **a web application developed in Python and structured in a multi-layer architecture**. The user interface is built using the Dash framework and communicates securely with a PostgreSQL database via a REST API built with Flask in the backend.

EEG Explorer's core functionality is supported by the Langchain framework, which **integrates OpenAI's ChatGPT4-Turbo advanced language model into the application**. Users write their question in natural language into the interface. It is then processed by artificial intelligence and translated into SQL queries to query the Spectre database. The result is in turn converted into natural language to display an intelligible answer for the user.

EEG Explorer facilitates access to complex EEG data and enables Spectre Biotech's managers to make more effective use of the company's data and tools. In future, this tool could also be offered to external users such as neuroscience researchers or healthcare professionals.

IV. CAHIER DES CHARGES

A. CONTEXTE

Dans le cadre de mon alternance en tant que data scientist chez Spectre Biotech nous avons décidé de réfléchir à un projet qui permettrait à la fois de mettre en œuvre mes compétences en data science, qui serait utile pour l'entreprise et qui me permettrait de passer la certification CDA. Spectre Biotech est une startup en phase de croissance, nous sommes peu de salariés pour beaucoup de travail. Afin de valoriser l'entreprise sans surcharger l'équipe technique nous avons pensé qu'**un outil permettant un requêtage simplifié de la base de données par le personnel non technique** aurait une forte valeur ajoutée (communication commerciale, publications scientifiques, ...) sans empiéter sur les missions déjà nombreuses de l'équipe technique, c'est ainsi qu'est né le projet EEG Explorer.

B. OBJECTIF

L'objectif du projet est de créer un outil qui permet de **poser des questions et d'obtenir des réponses en langage naturel** sur les données contenues dans la base de données Spectre.

EEG Explorer sera **une application web** accessible sur internet, orientée web, elle sera responsive pour s'adapter aux écrans desktop et mobiles. Elle doit respecter les règles de sécurité inhérentes à un tel projet pour **garantir la protection des données des utilisateurs et des données recueillies par Spectre**. Le RGPD est également pris en compte.

Sur le temps imparti, l'objectif est d'avoir un premier **prototype ou démonstrateur fonctionnel** (Proof of Concept - POC) contenant les fonctionnalités prioritaires afin de tester l'efficacité et l'utilité de l'application. D'autres fonctionnalités secondaires ont été listées mais il a été décidé de les développer plus tard.

C. FONCTIONNALITES PRIORITAIRES

Les fonctionnalités prioritaires à intégrer dans le premier POC sont les suivantes :

- **Inscription** : Permettre aux utilisateurs de créer un compte pour accéder à l'application.
- **Authentification** : Offrir la possibilité de se connecter et de se déconnecter, assurant ainsi une gestion sécurisée de l'accès utilisateur.
- **Interrogation de la Base de Données Spectre** : Proposer une fonctionnalité permettant aux utilisateurs d'interroger la base de données en langage naturel et de recevoir des réponses également en langage naturel (exemples en [annexe 1](#)).
- **Consultation de l'Historique** : Donner aux utilisateurs la possibilité de consulter et de télécharger leur historique de requêtes et de réponses pour un suivi facile de leurs interactions passées.
- **Gestion des Coordonnées Personnelles** : Permettre aux utilisateurs de consulter et de modifier leurs informations personnelles pour maintenir leur profil à jour.

- **Suppression de Compte** : Offrir une option pour supprimer définitivement le compte, en accord avec les normes de confidentialité et de sécurité des données.
- **Contact avec l'Équipe** : Fournir un moyen pour les utilisateurs de contacter l'équipe de développement ou de support pour des questions, des suggestions ou des retours.

D. PERIMETRE

Cette application est dans un premier temps **à destination des équipes non techniques** de Spectre Biotech en interne. A l'avenir, il sera envisagé d'ouvrir cette fonctionnalité au monde de la recherche et de la santé mais cela ne faisait pas partie du périmètre de ce premier POC.

E. RESSOURCES

L'application sera **créée 'from scratch'** afin de pouvoir être présentée pour la certification CDA, la **base de données** permettant la gestion des utilisateurs et le stockage de leurs historiques sera également créée de zéro.

L'objectif étant de ne pas surcharger l'équipe technique mais plutôt de la décharger, le projet sera **mené en autonomie** avec des revues de sprint pour s'assurer du respect du cahier des charges et arbitrer certaines décisions stratégiques. Sur le temps en entreprise, 1 jour par semaine est alloué au projet.

La **base de données Spectre** (à laquelle l'IA doit pouvoir se connecter) n'est pas accessible au moment où le projet est lancé. Sa création ne fait pas partie du projet EEG Explorer. Dans le cas où la **base de données** n'est pas prête au déploiement au moment voulu, une **base de données** simplifiée contenant un extrait de quelques centaines d'évaluations sera créée avec des données anonymisées ou créées aléatoirement.

F. PERFORMANCE

Pour évaluer l'efficacité d'EEG Explorer, nous utiliserons à la fois les **retours qualitatifs** des premiers utilisateurs et des **mesures quantitatives**. Parmi ces dernières, le taux d'utilisation de l'application, l'évolution du nombre de requêtes, et la fréquence des demandes d'assistance technique pour accéder aux données seront analysés. Nous suivrons aussi des **indicateurs spécifiques** à chaque requête : coût auprès d'OpenAI, temps d'exécution, et nombre de tokens utilisés. De plus, le **suivi des étapes intermédiaires du modèle** permettra de vérifier la fiabilité de ce dernier et de comprendre les éventuelles discordances entre les résultats obtenus et les attentes des utilisateurs, afin d'améliorer continuellement notre système.

V. GESTION DE PROJET

A. ORGANISATION ET TRAVAIL EN EQUIPE

Chez Spectre Biotech, l'équipe technique est composée d'un développeur full-stack, d'un lead data-scientist et de deux data scientists alternantes (dont moi). Le CEO de l'entreprise, initiateur du projet et utilisateur final était également concerné par la gestion de ce projet.

Pour mener à bien le projet EEG Explorer nous avons utilisé **une méthodologie Agile d'inspiration Kanban** adaptée à nos besoins, c'est ainsi que nous travaillons au quotidien sur l'ensemble des projets informatiques chez Spectre.

Les sprints de 3 semaines (le temps de travail alloué au projet était de un jour par semaine) permettaient de faire part des progrès et/ou des blocages éventuels rencontrés et de présenter régulièrement l'état d'avancement du projet à l'équipe tech. Le CEO se joignait à nous ponctuellement en fonction des avancées annoncées et des décisions à prendre.

Les semaines de formation à l'école (1 semaine/mois) étaient également mises à contribution pour présenter mon travail à mon formateur et à mes collègues de promotion et voir les projets de mes camarades et leur avancement. Ainsi nous pouvions échanger sur les bonnes pratiques de développement, les attendus pour la certification CDA, les solutions techniques testées et retenues et les lacunes restantes à combler.

Mon contrat d'alternance chez Spectre Biotech comme ma formation se sont passées en **distanciel à 100%**. Des deux côtés nous avons un serveur Discord comme canal de communication privilégié. Ainsi, **une communication continue** s'effectuait avec mes collègues et mon formateur pour partager les ressources, les idées et les conseils au fil du développement du projet.

Le code est rigoureusement commenté en anglais, avec **des docstrings détaillés pour chaque classe et fonction**, fournissant une explication claire de leur fonctionnement, des arguments attendus et des résultats éventuels. Des exemples sont inclus lorsque pertinents, et **des commentaires supplémentaires** sont ajoutés au besoin pour clarifier des points spécifiques. L'utilisation de **Visual Studio Code** comme environnement de développement intégré assure le respect des bonnes pratiques de codage grâce à l'utilisation de linters (flake8, mypy, pylint). Cette approche est essentielle pour **faciliter le travail en équipe**, simplifier la **maintenance du code** et assurer sa **compréhension et sa fiabilité à long terme**.

Le développement du projet EEG Explorer a été **versionné en local avec Git et en ligne avec GitHub**. Git est un système de contrôle de version distribué qui permet aux développeurs de suivre et de gérer les changements dans le code source au cours du temps. GitHub, une plateforme basée sur Git, facilite la collaboration et l'hébergement de code pour les projets informatiques. Chez Spectre Biotech nous utilisons **le modèle Gitflow** (voir figure 1), c'est une approche structurée de Git qui définit un ensemble de branches pour segmenter le travail selon les différentes phases de développement. Même en étant la seule développeuse du projet j'ai tout de même respecté a minima le workflow habituel de Spectre Biotech avec Git afin de m'habituer et d'automatiser **les bonnes pratiques de développement**.



Figure 1 : Illustration du workflow Gitflow

Comme le montre le schéma ci-dessus, avec gitflow il existe plusieurs branches principales : 'Master' pour les versions de production, 'Develop' pour les développements en cours, 'Feature' pour les nouvelles fonctionnalités, 'Release' pour les préparations de nouvelles versions, et 'Hotfix' pour les correctifs d'urgence. Les nouvelles fonctionnalités sont développées dans des branches 'Feature', puis fusionnées dans 'Develop'. Une fois prêtes pour une nouvelle version, elles sont fusionnées dans une branche 'Release' pour les tests finaux, puis dans 'Master' et marquées avec un numéro de version. Les correctifs d'urgence ('Hotfix') sont directement appliqués sur 'Master' et intégrés à 'Develop', assurant que les corrections importantes sont rapidement déployées et intégrées dans le flux de travail de développement.

Pour ma part, à partir du moment où une première version minimale de l'application a été conçue, j'ai créé la branche 'Develop' et, à partir d'elle, **une branche pour chaque feature ou ensemble de feature que je développais**. Une fois la feature terminée et testée, elle était fusionnée dans 'Develop'.

```
# Create the 'Develop' branch from the 'Master' branch
git checkout master
git pull origin master # Ensure we have the latest version of 'Master'
git checkout -b develop
git push -u origin develop

# Create a new 'feature' branch from 'Develop' for the new functionality
git checkout develop
git checkout -b feature/add-ai-agent

# Development of the functionality, in this case adding an AI agent
git add {filename} {filename}
git commit -m "commit description"

# After completing the development and testing of the feature
git pull origin develop # Incorporate the latest changes from 'Develop' if necessary
git checkout develop
git merge --no-ff feature/add-ai-model # Merge the feature branch into 'Develop'
git push origin develop # Push the changes to the remote server
```

B. PLANIFICATION DES ACTIVITES

La gestion des tâches a été organisée via l'outil de tableau Kanban sur Notion, cet outil de planification m'a permis d'**organiser mon travail sous la forme d'un backlog**. J'ai créé 4 états de carte : Idées, Next, En cours et Terminée. Dès que j'avais une idée de fonctionnalité ou de tâche à réaliser, ou que mes interlocuteurs m'en proposaient une nouvelle, je créais une carte dans la section Idées.

Chaque carte contient un titre explicite sur la tâche à faire, un thème (Front, API, BDD, IA ...) et une section de commentaires. Pour chaque carte, je fixais une date de rendu afin de pouvoir mesurer l'avance ou le retard pris dans mon travail et m'adapter en conséquence.

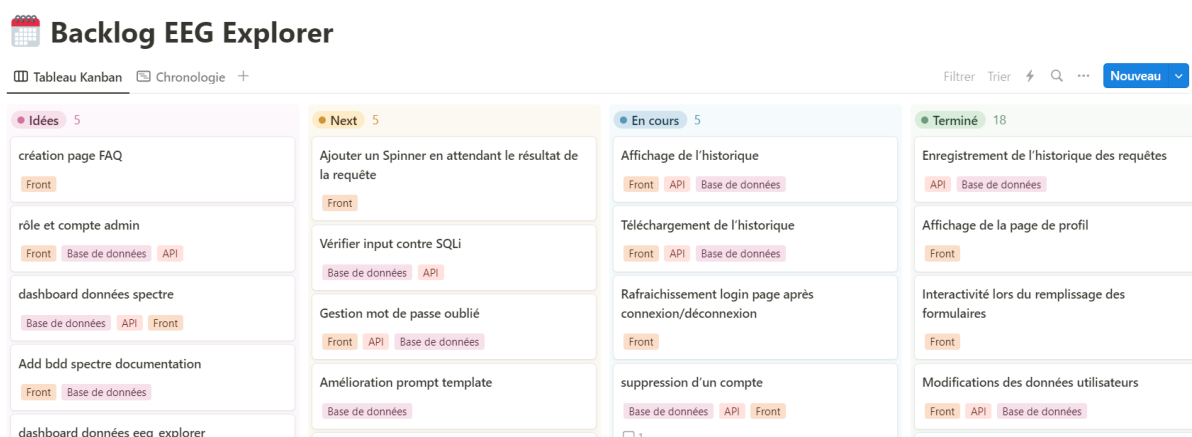


Figure 2 : Capture d'écran du backlog sur Notion

Au début de chaque session de travail j'**organisais les prochaines étapes de mon travail** en déplaçant les cartes dans les colonnes Next ou En cours. A la fin de chaque session, je pouvais revenir sur mon backlog pour **étiqueter les activités terminées** et ajouter des commentaires dans les cartes si nécessaire. Lors des réunions de revue de sprint, je créais de nouvelles cartes en fonction des idées discutées et je réorganisais mon travail en fonction des retours reçus.

La conception et le développement d'une application « from scratch » est **difficile à évaluer d'un point de vue technique, temporel et humain, surtout pour une développeuse junior**. Au fil des apprentissages de la formation et du travail en entreprise, le planning du projet et l'alimentation du backlog ont évolué et ont **nécessairement demandé des adaptations**. Le déroulé temporel prévisionnel du projet EEG Explorer est disponible en [annexe 2](#).

VI. SPECIFICATIONS FONCTIONNELLES

Le projet EEG Explorer est avant tout basé sur **une approche exploratoire visant à découvrir les possibilités offertes par les technologies émergentes de l'intelligence artificielle**. Les spécifications fonctionnelles ont été listées lors du lancement du projet mais peuvent être amenées à évoluer pour proposer de nouvelles pistes à explorer au-delà des besoins initiaux.

A. ATTENTES PRINCIPALES

Un outil déployé en ligne : une application internet à laquelle l'utilisateur peut se connecter via une url pour accéder au service.

Un outil simple et intuitif : un outil à destination des personnels non techniques, facile à prendre en main, qui ne demande pas d'effort d'apprentissage pour les utilisateurs mais leur donne accès simplement et de manière intelligible à des informations non accessibles auparavant.

Un outil fiable et sécurisé : une solution qui apporte des réponses fiables sans mettre en péril la sécurité et la protection des données.

B. CAS D'USAGE

La première étape après avoir défini le cahier des charges a été **d'identifier les différents cas d'utilisation de l'application**, ils sont illustrés dans la figure ci-dessous.

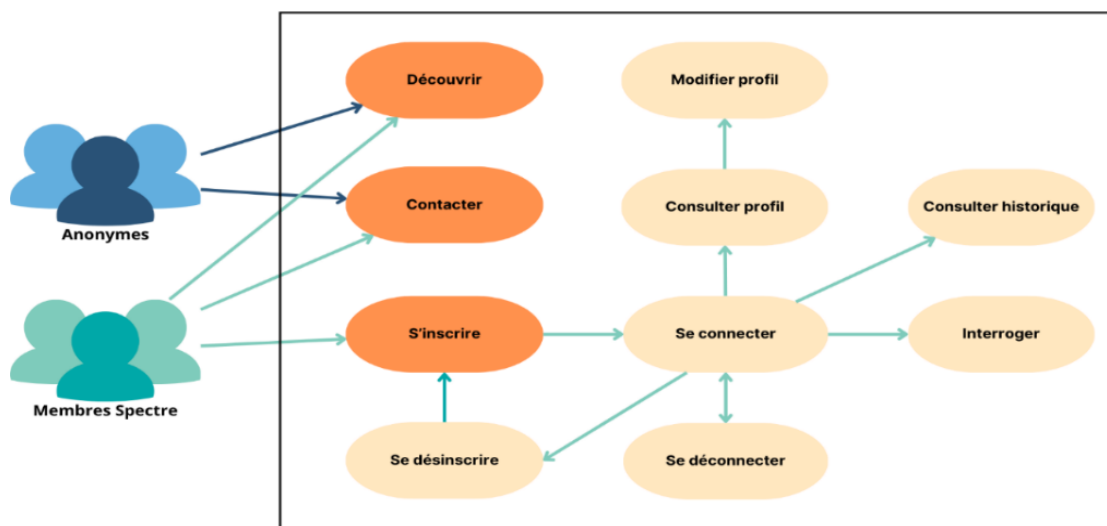


Figure 3 : Diagramme des cas d'utilisation générale

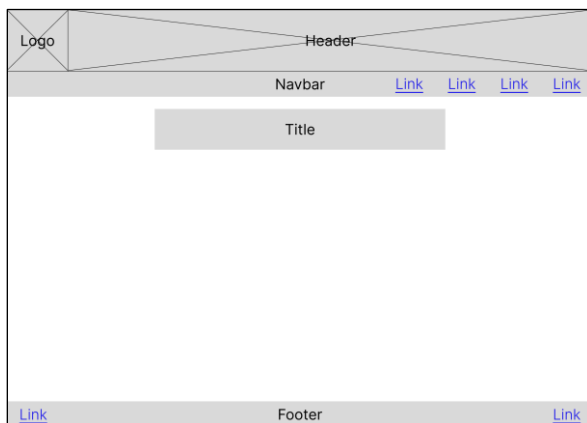
Les visiteurs anonymes ont la capacité de découvrir l'application et d'utiliser le formulaire de contact. Les membres affiliés à Spectre Biotech bénéficient, en plus, de l'option d'inscription, leur ouvrant la porte à l'ensemble des fonctionnalités une fois connectés. Ces services incluent l'exploration interactive de la **base de données Spectre**, la consultation et la gestion de leur historique de requêtes, ainsi que la revue et la modification de leur profil. Ils disposent également des fonctionnalités de déconnexion et de désinscription, pour un contrôle complet de leur engagement avec l'application.

C. CONCEPTION GRAPHIQUE ET FONCTIONNELLE

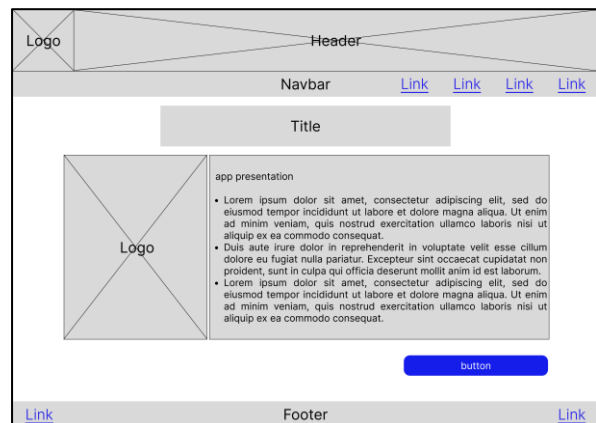
Dans le cadre du projet EEG Explorer, j'ai réalisé l'ensemble des étapes de conception graphique et fonctionnelle de l'application telle que la **création de la charte graphique, des wireframes et des maquettes**.

À partir d'une analyse détaillée du cahier des charges et en m'appuyant sur les uses cases, j'ai pu **concevoir les premiers wireframes sur Figma**. Cette étape initiale a permis de poser les bases de l'interface et de l'expérience utilisateur, en mettant l'accent sur la simplicité et l'efficacité (l'ensemble des wireframes est disponible en [annexe 3](#)).

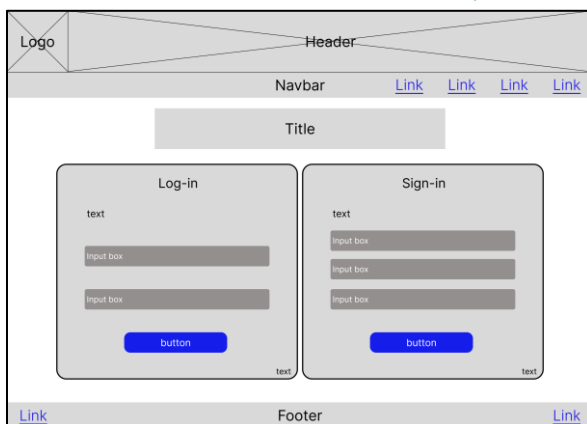
Figure 4 : Exemples de wireframes



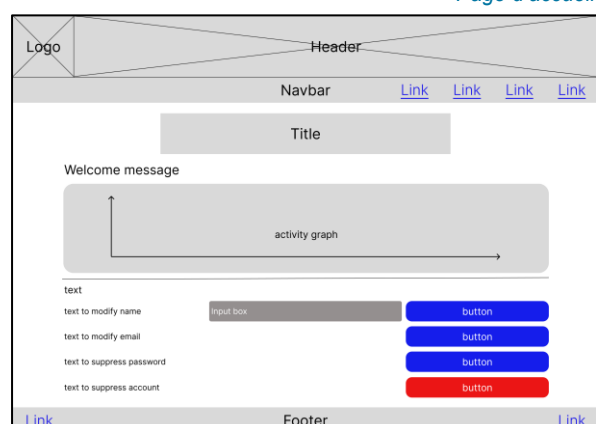
Template du site



Page d'accueil



Page d'inscription et de connexion



Page de profil

En parallèle, pour la charte graphique, la demande des décideurs était de **créer une identité propre à EEG Explorer** en gardant la couleur principale du logo Spectre et la typographie du site internet. J'ai donc commencé par créer une **palette de couleur** avec le site coolors.co avec des nuances de bleu jusqu'au blanc. A partir de cette palette j'ai demandé à DALL-E (l'intelligence artificielle de création d'images d'OpenAI) de **créer le logo** qui servira aussi de favicon pour le site. J'ai également modifié la colorimétrie d'une image d'archive pour créer l'en-tête de l'application. En complément de **la typographie** Inter de Google fonts, utilisée pour tous les textes de l'application, j'ai également choisi la typographie Nort d'Adobe fonts pour les titres et sous-titres, une typographie ressemblante à celle du logo de Spectre Biotech. Au fil de la construction de la maquette, quand des images étaient nécessaires, elles ont

également été créées par IA sur la base de la palette de couleur (l'ensemble de la charte graphique est visible en [annexe 4](#)).

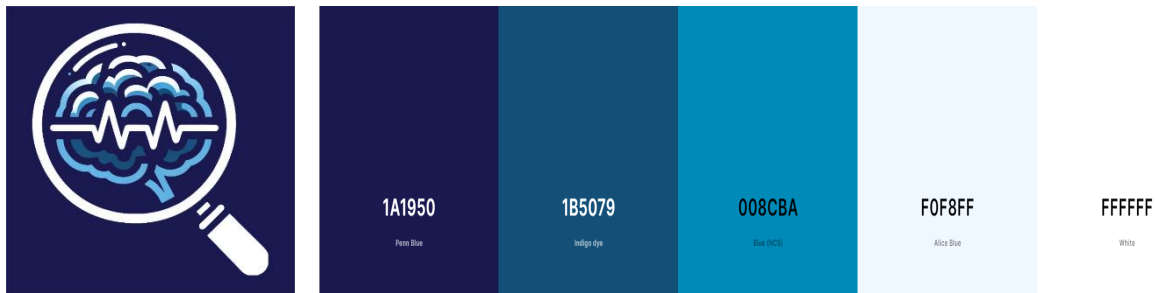
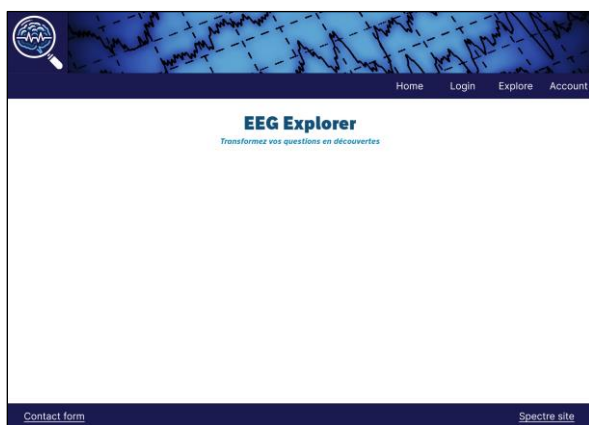


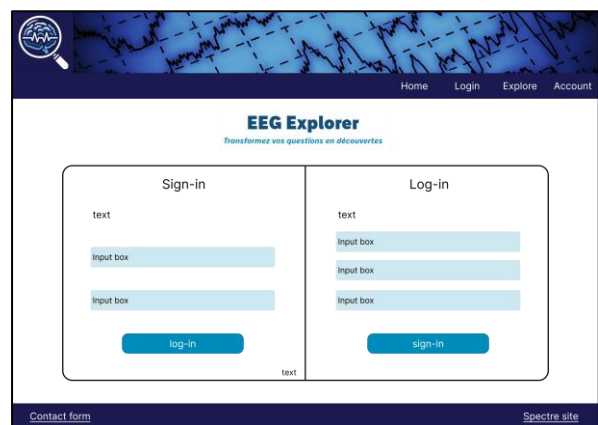
Figure 5 : Logo et palette de couleurs définis dans la charte graphique

Une fois les wireframes et la charte graphique élaborés, j'ai présenté ces éléments à mes employeurs pour validation. Leurs retours constructifs ont été intégrés dans la version finale, permettant ainsi de passer à l'étape suivante : **la réalisation des maquettes**. Celles-ci ont été conçues pour offrir un aperçu plus concret de l'application, en incorporant les éléments validés précédemment. L'étape des maquettes a été cruciale pour finaliser la conception visuelle et fonctionnelle de l'application. En utilisant l'outil **prototype de Figma**, j'ai pu **simuler l'interaction utilisateur**, ce qui a permis aux décideurs de visualiser concrètement le produit final et comment les utilisateurs interagiraient avec l'application (l'ensemble des maquettes est visible en [annexe 5](#)).

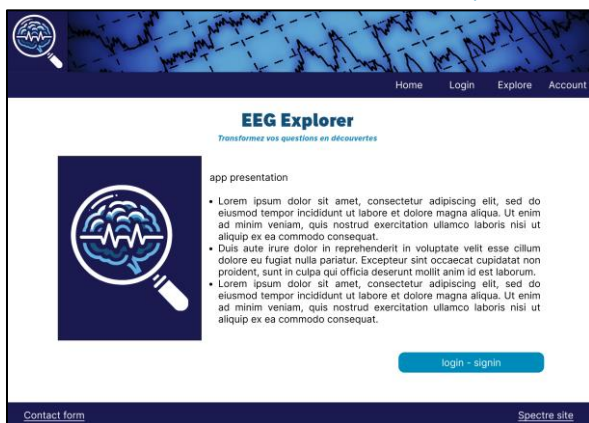
Figure 6 : Exemples de maquettes



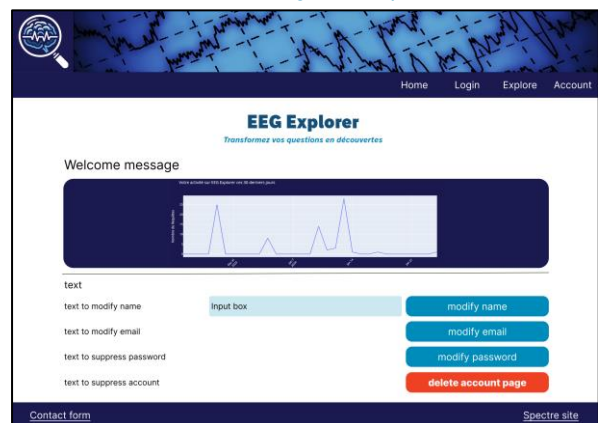
Template du site



Page d'inscription et de connexion



Page d'accueil



Page de profil

D. ARBORESCENCE FONCTIONNELLE POUR L'UTILISATEUR

La figure ci-dessous représente l'architecture fonctionnelle de l'application pour un utilisateur. Chaque page et chaque fonctionnalité y apparaît afin de **mieux appréhender le parcours utilisateur**. Chaque page est accessible en 3 clics ou moins à partir de la page d'accueil.

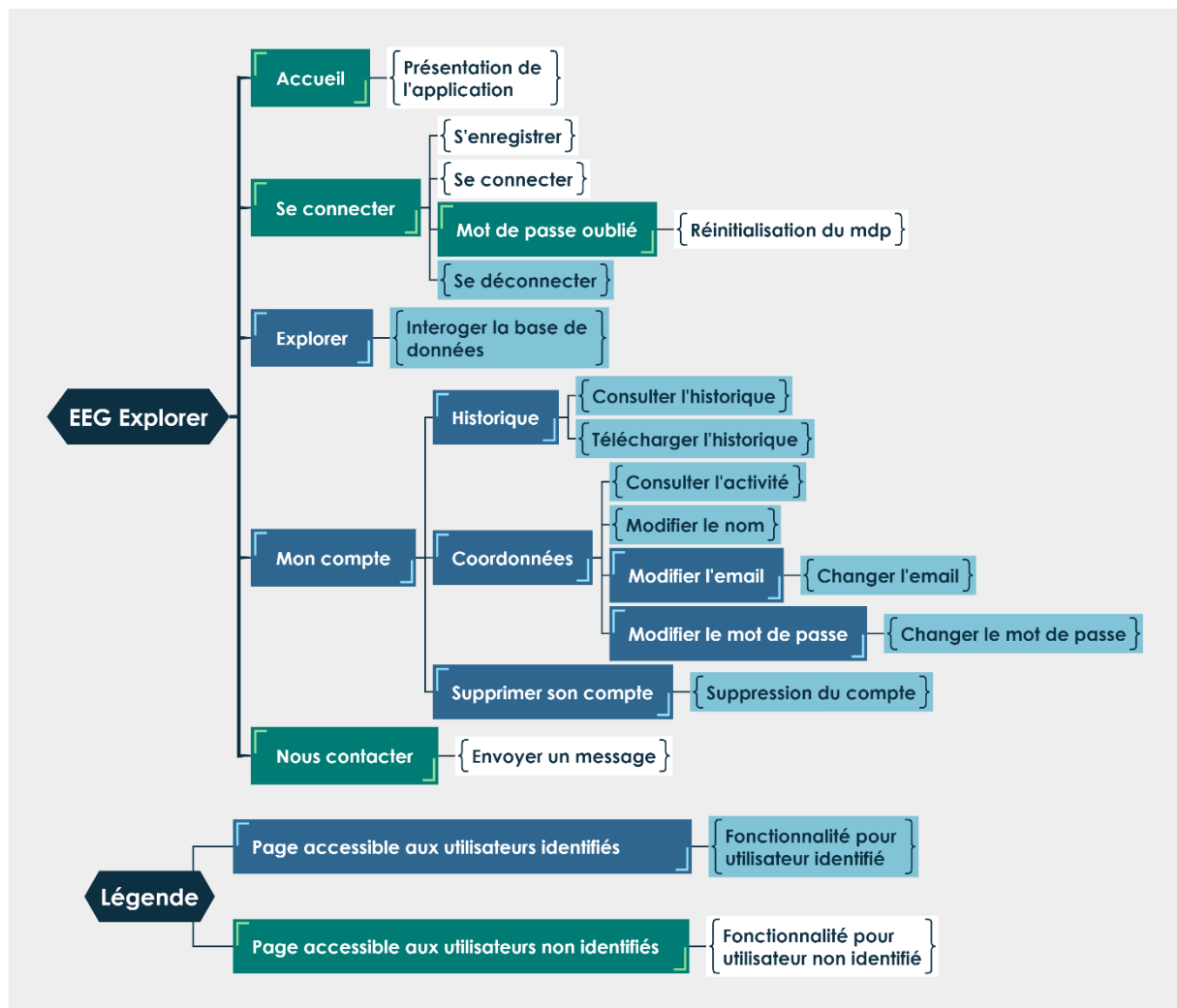


Figure 7 : Architecture fonctionnelle de l'application EEG Explorer

VII. SPECIFICATIONS TECHNIQUES

A partir des spécifications fonctionnelles décrites dans la partie précédente, j'ai pu définir les spécifications techniques du projet EEG Explorer et son architecture.

A. ENVIRONNEMENT

Pour développer le projet EEG Explorer, **Poetry a été adopté comme outil de gestion des dépendances et des packages**. Il permet de déclarer les bibliothèques dont dépend le projet Python dans un fichier unique `pyproject.toml` (voir ci-dessous), ce qui **facilite la gestion des versions de packages et des environnements virtuels associés**. Poetry s'occupe automatiquement de la résolution des dépendances et peut construire et publier le paquet final de manière cohérente. Grâce à son système de gestion de dépendances précis, il assure que les installations sont reproductibles, et donc que **le projet peut être installé de la même manière sur n'importe quel système**. En utilisant Poetry, l'environnement de développement du projet est standardisé et les incohérences entre les environnements minimisées. Ce qui permettra **une intégration et un déploiement continu efficaces** pour l'application EEG Explorer.

```
[tool.poetry.dependencies]
python = "^3.12.0"
dash = "^2.14.1"
numpy = "^1.26.2"
pandas = "^2.1.3"
psycpg2-binary = "^2.9.9"
flask-login = "^0.6.3"
flask-sqlalchemy = "^3.1.1"
passlib = "^1.7.4"
python-dotenv = "^1.0.0"
argon2-cffi = "^23.1.0"
dash-bootstrap-components = "^1.5.0"
langchain = "^0.0.343"
openai = "^1.3.6"
openpyxl = "^3.1.2"
stringgenerator = "^0.4.4"
plotly = "^5.18.0"

[tool.poetry.group.dev.dependencies]
pytest = "^8.0.0"
```

B. INTERFACE UTILISATEUR

Le framework open source **Dash** est utilisé pour la création de **l'interface utilisateur web**. La création de pages **HTML** avec Dash s'appuie sur une approche déclarative en **Python**. Concernant le style des pages HTML, j'ai utilisé **Bootstrap** pour gérer le **CSS**, ce qui me permet d'intégrer facilement des styles et **des composants d'interface utilisateur réactifs et responsives**. Bootstrap offre un large éventail de modèles CSS préconçus, rendant la personnalisation de l'apparence et la mise en page de l'application Dash rapide et efficace.

C. BACKEND ET BASE DE DONNEES

La liaison entre le front et le back se fait avec **les callbacks de Dash**. Ce sont des fonctions Python qui relient les interactions utilisateur sur l'interface graphique, comme des clics ou des entrées de texte, à des actions de back-end, permettant ainsi une mise à jour dynamique et interactive des composants de l'application en fonction des entrées de l'utilisateur. Cela permet de **développer l'ensemble de l'application en Python** et d'utiliser **la programmation orientée objet**.

J'ai créé **une API avec Flask** pour recevoir les requêtes HTTP envoyées par le back-end et accéder à la **base de données EEG Explorer**. **SQLAlchemy est utilisé comme ORM (Object-Relational Mapping)** pour faciliter l'interaction entre les scripts Python et la **base de données**.

La **base de données est conçue avec PostgreSQL**, un système de gestion de base de données (SGBD) relationnelle qui permet de développer tout l'environnement de la **base de données**. Le **logiciel PgAdmin** permet d'administrer la **base de données**.

D. AUTHENTIFICATION

Flask-Login est une **extension pour Flask**. Je l'ai utilisée pour la gestion de l'authentification et des sessions utilisateurs dans EEG Explorer. Son fonctionnement repose sur l'utilisation des "user loaders", des **fonctions qui servent à charger un utilisateur à partir d'un identifiant**. Lorsqu'un utilisateur se connecte, une fois ses identifiants vérifiés, Flask-Login stocke son identifiant dans la session du navigateur, permettant ainsi une **récupération aisée de l'utilisateur actif à chaque requête**. Flask-Login fournit aussi des **fonctionnalités pour la déconnexion** et des décorateurs utiles comme **@login_required**, qui restreint l'accès aux vues aux seuls utilisateurs authentifiés.

E. TRAITEMENT DU LANGAGE NATUREL & IA

Langchain, un framework axé sur l'intégration d'agents conversationnels avancés dans les applications web, fournit des outils adaptés pour faciliter la connexion de ces modèles à des sources de données personnalisées. Cela permet des réponses contextuelles. Il est donc particulièrement utile pour créer des interfaces conversationnelles à partir de bases de données complexes comme nous voulions le faire. En utilisant les bibliothèques et les modèles de Langchain j'ai développé un **agent SQL spécialisé pouvant accéder à la base de données Spectre**.

Cet agent dispose d'outils spécifiques à son rôle lui permettant de :

- Comprendre la question de l'utilisateur,
- Scanner le schéma de la **base de données Spectre** et ses tables,
- Convertir la question en requête SQL, tester cette requête et, si elle est correcte, l'exécuter
- Analyser le résultat pour retranscrire une réponse finale à l'utilisateur en langage naturel.

Pour des **raisons d'accessibilité et de performances**, nous avons choisi d'utiliser le modèle **ChatGPT-4 Turbo d'OpenAI**. Nous envisageons également à l'avenir d'explorer d'autres pistes, telles que les modèles open-source de Meta et de Mistral.

F. ARCHITECTURE

L'application EEG Explorer est composée de plusieurs couches, la figure 2 illustre les différents composants de l'application et leurs interactions ainsi que les technologies utilisées.

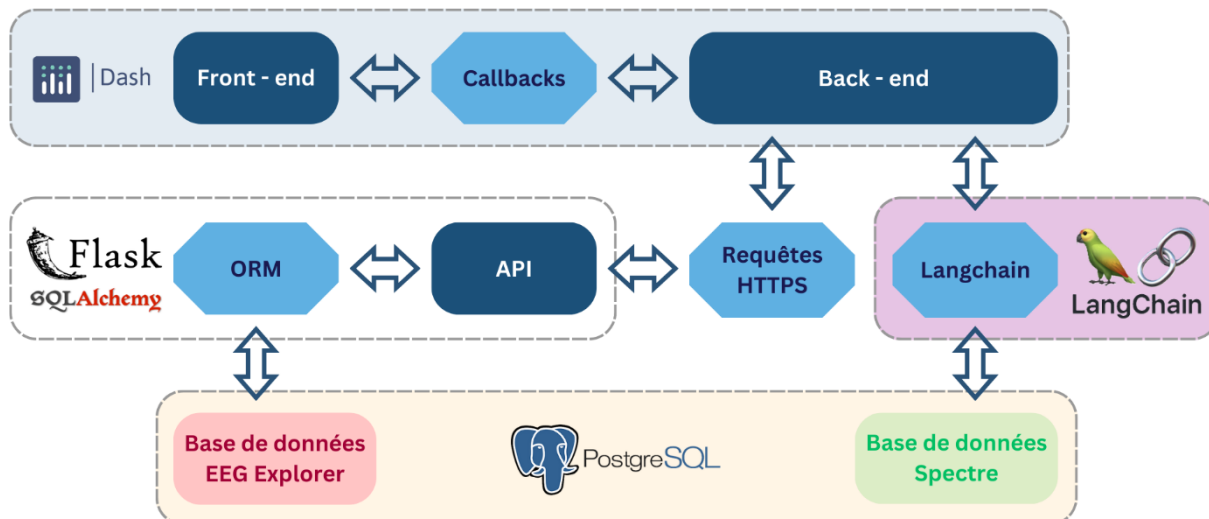


Figure 8 : Architecture multicouche et technologies principales d'EEG Explorer

G. SECURITE ET CONFORMITE

Des mesures de sécurité, y compris la **gestion sécurisée des sessions utilisateur** et la **protection contre les vulnérabilités courantes du web (injections SQL, attaques XSS, ...)**, sont implémentées conformément aux bonnes pratiques en vigueur.

La sécurisation des données et des bases de données s'effectue à tous les niveaux de l'application. On retrouvera donc **des restrictions sur le front-end**, **des vérifications dans les scripts python sur le back-end** et les routes API et **des interdictions ou limitations de droits sur les bases de données**. La sécurité d'EEG Explorer passe aussi par le déploiement conteneurisé par Docker.

H. DEPLOIEMENT

Lorsque EEG Explorer sera prête pour le déploiement, nous prévoyons de l'héberger sur le même serveur Linux (Ubuntu) que celui utilisé pour la plateforme Spectre Biotech. Ce serveur accueillera également la **base de données Spectre**, à laquelle nous nous connecterons via LangChain.

Le déploiement d'EEG Explorer comprendra plusieurs étapes essentielles :

Création des Dockerfile : Chaque composant du projet (API Flask, application Dash, **base de données** PostgreSQL) disposera de son propre Dockerfile. Ces fichiers contiendront les instructions nécessaires pour construire des images Docker spécifiques à chaque composant. Pour PostgreSQL, nous utiliserons une image Docker officielle. Pour l'API Flask et l'application Dash, les Dockerfile incluront des directives pour installer Python, copier les sources, configurer Poetry pour gérer les dépendances (comme indiqué dans le fichier pyproject.toml), et définir la commande de lancement de l'application. Concernant l'API Flask, nous ajouterons également une commande pour démarrer Flask avec un serveur WSGI, Gunicorn dans notre cas. Pour l'application Dash, cette étape est gérée automatiquement en interne.

Configuration avec docker-compose.yml : Un fichier docker-compose.yml sera créé à la racine du projet pour orchestrer l'interaction des conteneurs Docker. Ce fichier définira trois services : un pour l'API Flask, un pour l'application Dash, et un pour la **base de données** PostgreSQL. Nous veillerons à ce que les ports sur lesquels Gunicorn écoute soient correctement mappés pour permettre une communication externe.

Lancement des Conteneurs : L'exécution de « docker-compose up » commandera à Docker de démarrer les conteneurs selon les configurations établies. Les images des composants seront construites ou téléchargées (pour PostgreSQL) à partir des Dockerfiles. Il sera crucial de tester l'application en s'assurant que l'accès à l'application Dash (front-end) se fait sans entrave, et que la communication avec les autres couches de l'application (API et base de données) est opérationnelle.

Installation sur le Serveur : Après connexion SSH au serveur, Docker sera installé, activé et lancé. Le projet sera ensuite cloné depuis GitHub, et « docker-compose up » sera exécuté dans le répertoire cloné pour démarrer les conteneurs Docker.

Configuration du Serveur Web Nginx : Notre serveur web Nginx, déjà installé, sera configuré comme un proxy inverse pour rediriger les requêtes vers les conteneurs Docker via le serveur WSGI. Le DNS sera ajusté pour diriger vers l'adresse IP du serveur.

Sécurisation : Un pare-feu sera configuré et un certificat SSL, obtenu via [Certbot](#), sera installé pour renforcer la sécurité. Ceci permettra également l'activation du protocole HTTPS, assurant une communication sécurisée et chiffrée entre les utilisateurs et l'application.

Tests Finaux : Enfin, des tests complets seront effectués en accédant à l'application via le nom de domaine dans un navigateur web pour s'assurer du bon fonctionnement de tous ses composants.

Ce processus méthodique permettra le déploiement efficace de l'application EEG Explorer sur un serveur web, la rendant accessible en toute sécurité sur Internet. En fonction des éventuelles évolutions qui impacteraient le projet EEG Explorer nous pourrions être amenés à revoir le plan initialement conçu pour le déploiement afin de l'adapter aux besoins du projet si nécessaire.

VIII. REALISATIONS SIGNIFICATIVES

A. UN FRONT-END RESPONSIVE ET ACCESSIBLE

La réalisation du front-end dans une application Dash s'effectue principalement à travers l'utilisation des 'components' Dash, HTML et Bootstrap, qui offrent ensemble une gamme étendue d'éléments d'interface utilisateur. Les **Dash Core Components (dcc)** sont des éléments interactifs personnalisables tels que des graphiques, des sliders, et des dropdowns, permettant une intégration facile des fonctionnalités de visualisation de données. Les **HTML Components**, quant à eux, fournissent des wrappers Python pour toutes les balises HTML standards, ce qui facilite la structuration du layout de l'application avec des éléments familiers comme div, span, et h1. En complément, les **Bootstrap Components**, disponibles via la bibliothèque Dash Bootstrap Components (dbc), permettent d'incorporer des éléments de Bootstrap, framework de front-end reconnu, pour un design réactif et fluide. Cette intégration de différents types de composants permet de créer des interfaces riches et interactives, tout en bénéficiant de la simplicité et de la flexibilité du code Python.

Concernant plus particulièrement la mise en page du site, je me suis appuyée sur le **système de grille de Bootstrap** et sur ses trois composants principaux : Container, Row et Column. **Un Container est l'élément englobant qui centre le contenu à l'écran**, servant de canevas pour la mise en page du site. À l'intérieur d'un Container, **des Rows sont utilisées pour définir des lignes horizontales**, et chaque Row peut contenir plusieurs Columns. Ces Columns sont les blocs de construction qui permettent de diviser l'espace horizontal de chaque Row en sections. **Ce système de grille est basé sur un total de 12 unités par ligne**, permettant une grande flexibilité pour spécifier la largeur de chaque Column. En ajustant la largeur des Columns à l'intérieur d'une Row, on peut créer des **designs adaptatifs qui réagissent de manière fluide à différentes tailles d'écran**, assurant ainsi que le contenu s'affiche de manière optimale sur des dispositifs allant des téléphones mobiles aux grands écrans de bureau. Cela rend le design web accessible et esthétiquement agréable sur différents dispositifs, sans nécessiter de multiples versions du site.

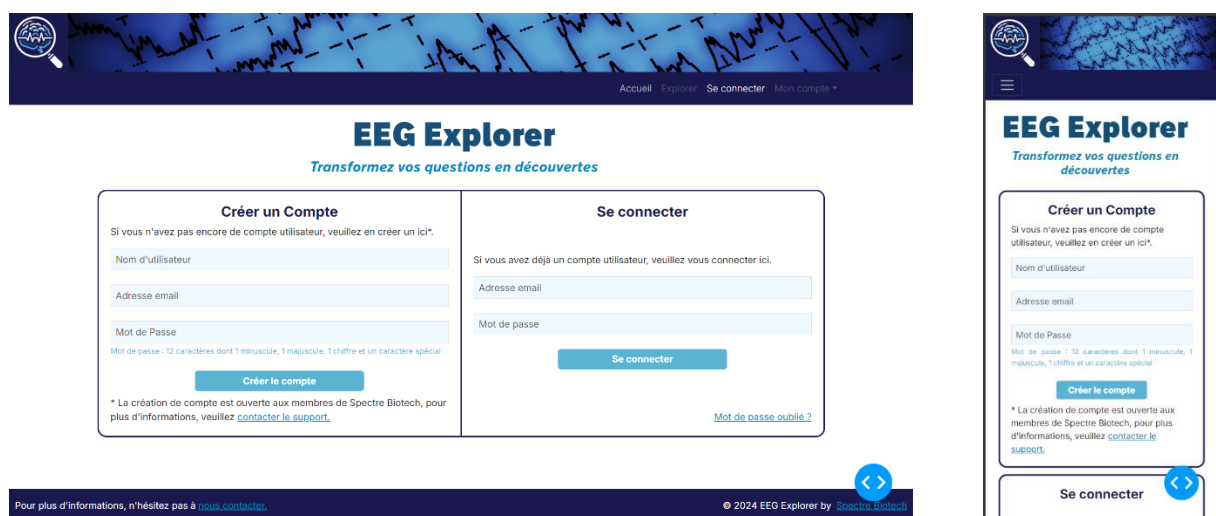


Figure 9 : Rendu responsive de la page d'inscription et de connexion

```

login_page_layout = dbc.Container(
    [
        *title_container, # Rows with title and subtitle
        dbc.Row(
            [
                dbc.Col(
                    signup_card,
                    width=12, md=6, lg=5,
                ),
                dbc.Col(
                    login_card,
                    width=12, md=6, lg=5,
                ),
            ],
            justify="center",
        ),
    ],
    className="py-3",
    fluid=True,
)

```

L'exemple de la page d'inscription et de connexion d'EEG Explorer démontre **l'utilisation pratique du framework Bootstrap dans la conception du front-end**. Le `dbc.Container` sert de conteneur principal pour les éléments de la page, avec la propriété `fluid` réglée sur `True`, permettant ainsi au contenu de **s'adapter automatiquement à la taille de l'écran de l'utilisateur**. La mise en page utilise des `dbc.Card` pour les sections de connexion et d'inscription, disposées au sein d'une **grille structurée** avec `dbc.Row` et `dbc.Col`. Cette structure assure une organisation claire des éléments. Les paramètres `width=12`, `md=6`, `lg=5` définissent la réactivité de la mise en page. Par défaut chaque carte occupe toute la largeur (12 colonnes sur 12) et sont donc l'une en dessous de l'autre. Sur un écran de taille moyenne (`md`) ou plus, elles se répartissent sur deux colonnes de 6, se plaçant côte à côte. Enfin, sur les écrans larges (`lg`) ou plus, chaque carte occupe 5 colonnes, permettant un alignement centralisé avec un espace de chaque côté, grâce à `justify="center"` appliqué à `dbc.Row`. Cette approche assure une **expérience utilisateur cohérente et esthétiquement agréable sur différents appareils**, qu'il s'agisse d'écrans de petite taille ou de moniteurs plus larges.

```

def make_form(process: str, item: str, item_type: str, placeholder: str) -> list:
    """ Create a form part.
    Args:
        process (str): The process to which the form part belongs.
        item (str): The item to be created (e.g. username, password, etc.)
        item_type (str): The type of the item (e.g. text, password, etc.)
        placeholder (str): The placeholder for the item.
    Returns:
        list: A list containing the form elements.
    """
    form_elements = [dbc.Input(
        id=f'{process}-{item}', type=item_type, placeholder=placeholder
    ),
        dbc.FormFeedback(type="valid", id=f'{item}-feedback-valid'),
        dbc.FormFeedback(type="invalid", id=f'{item}-feedback-invalid')]
    return form_elements

```

La création des champs du formulaire est automatisée par la fonction **make_form** (voir le code page précédente) qui génère des `dbc.Input` pour les saisies utilisateur, ainsi que des `dbc.FormFeedback` pour l'affichage de la validation des données.

```
signup_card = dbc.Card(
    [
        html.H2("Créer un Compte"),
        html.P(
            "Si vous n'avez pas encore de compte utilisateur, ...",
            className="me-auto"
        ),
        *make_form('signup', 'name', 'text', 'Nom d\'utilisateur')
        *make_form('signup', 'email', 'email', 'Adresse email')
        *make_form('signup', 'password', 'password', 'Mot de passe')
        dbc.FormText(
            "Mot de passe : 12 caractères dont 1 minuscule, 1 majuscule, ...",
            id='password-feedback',
            className="me-auto mb-3",
            color='info',
        ),
        dbc.Button(
            "Créer le compte",
            id='signup-button',
            className="w-50 my-3",
            disabled=True,
        ),
        html.Div(
            children=[
                "* La création de compte est ouverte aux membres de Spectre ...",
                html.A(
                    "contacter le support.",
                    href="/contact",
                    target="_blank",
                ),
            ],
            id='signup-status',
            className="mb-auto",
        ),
    ],
    id="signup-card",
    class_name="common-card-style",
)
```

Chaque élément du formulaire (code ci-dessus) est identifié de manière unique pour une interaction fluide avec les callbacks de Dash et adopte des classes Bootstrap standard pour l'uniformité du style. Le bouton de soumission, un `dbc.Button`, initialement désactivé, est programmé pour devenir interactif une fois que les champs de saisie passent la validation effectuée par les callbacks.

En complément, des éléments HTML natifs comme `html.H2` et `html.A` sont intégrés pour enrichir la structure du contenu. L'application de ces techniques dans Dash, en conjonction avec Bootstrap, réalise une interface de formulaire cohérente et réactive, où **le feedback est immédiatement communiqué à l'utilisateur** en réponse à ses actions.

Dash est construit sur React.js, à partir du code python les composants sont convertis en leurs équivalents React et c'est React.js qui gère la génération du DOM HTML coté client. Ci-dessous, voici le code HTML généré par le script Python pour le formulaire d'inscription.

```
<div class="common-card-style card">
  <h2>Créer un Compte</h2>
  <p class="me-auto">Si vous n'avez pas encore de compte utilisateur, veuillez en créer un ici*.</p>
  <input type="text" class="form-control mt-3" id="signup-name" placeholder="Nom d'utilisateur" step="any">
  <div id="username-feedback-valid" class="valid-feedback"></div>
  <div id="username-feedback-invalid" class="invalid-feedback"></div>
  <input type="email" class="form-control mt-3" id="signup-email" placeholder="Adresse email" step="any">
  <div id="email-feedback-valid" class="valid-feedback"></div>
  <div id="email-feedback-invalid" class="invalid-feedback"></div>
  <input type="password" class="form-control mt-3" id="signup-password" placeholder="Mot de Passe" step="any">
  <div id="password-feedback-valid" class="valid-feedback"></div>
  <div id="password-feedback-invalid" class="invalid-feedback"></div>
  <small id="password-feedback" class="me-auto text-info form-text">Mot de passe : ...</small>
  <button disabled="" id="signup-button" class="w-50 btn btn-primary my-3">Créer le compte</button>
  <div id="signup-status" class="mb-auto">* La création de compte ...
    <a href="/contact" target="_blank" previewlistener="true">contacter le support.</a>
  </div>
</div>
```

L'**accessibilité** est une composante essentielle du développement front-end, assurant que l'application est utilisable par un large éventail d'utilisateurs, y compris **ceux ayant des besoins spécifiques**. Pour cela, nous avons intégré des **textes de remplacement descriptifs pour toutes les images**, permettant aux lecteurs d'écran de fournir un contexte aux utilisateurs malvoyants. En termes de design visuel, nous avons **soigneusement sélectionné des contrastes et des couleurs**, en veillant à ce qu'ils soient suffisamment distincts pour être facilement distinguables, même pour les **personnes rencontrant des difficultés de vision des couleurs**. Des tests ont été effectués sur le site de création de la palette de couleur coolors.co avec l'option Color Blindness et avec l'extension Chrome [Colorblindly](#). On peut voir ci-dessous le rendu de la page d'accueil d'EEG Explorer pour une personne souffrant de daltonisme par absence de vision du bleu (Tritanopie).



Figure 10 : Rendu de la page d'accueil en cas de daltonisme

B. UNE INTERFACE INTERACTIVE

Dans Dash, **les callbacks servent de pont entre le front-end et le back-end**. Ils fonctionnent comme des points de connexion qui écoutent les interactions sur le front-end (comme les clics ou les entrées de l'utilisateur) et déclenchent des réponses côté serveur. Un callback est composé de trois éléments clés :

- **Les Outputs** : Ce sont les éléments du front-end qui seront mis à jour ou modifiés en réponse à une interaction. Ils définissent le "quoi" de la réponse.
- **Les Inputs** : Ce sont les déclencheurs du callback. Ils spécifient "quand" le callback doit s'exécuter, souvent en réponse à une action de l'utilisateur, comme un clic sur un bouton.
- **Les States** : Ils représentent l'état actuel d'autres éléments de l'interface utilisateur qui peuvent être nécessaires pour exécuter le callback, mais qui ne le déclenchent pas directement.

Le cœur du callback est **une fonction Python** qui définit la logique de traitement des données entrantes (Inputs et States) et détermine ce qui doit être renvoyé au front-end (Outputs). Cette fonction est exécutée automatiquement chaque fois que les Inputs sont activés, permettant **une interaction dynamique et réactive** entre l'utilisateur et l'application Dash.

```
@app.callback(
    [Output("signup-email", "valid"),
     Output("signup-email", "invalid"),
     Output("email-feedback-valid", "children"),
     Output("email-feedback-invalid", "children")],
    [Input("signup-email", "value")],
)
def check_email_validity(
    email: str
) -> tuple[bool, bool, str, str]:
    """ Check if the email is safe, valid and allowed
    Args:
        email (str): email to check
    Returns:
        bool: is email valid
        bool: is email invalid
        bool: valid feedback
        bool: invalid feedback """
    if not email:
        return False, False, '', ''
    if not is_email_valid(email) or not is_input_safe(email):
        return False, True, '', 'Ceci n\'est pas une adresse email valide.'
    if not is_email_allowed(email):
        return False, True, '', 'Adresse email non autorisée.'
    return True, False, '', ''
```

Par exemple, le callback ci-dessus gère la validation de l'adresse email saisie par l'utilisateur. Dès que la saisie commence, la fonction **check_email_validity** est appelée. Cette fonction évalue si l'adresse email est dans un format correct, sans risque d'injection SQL (via la fonction **is_input_safe**) et si elle est autorisée via d'autres fonctions codées par ailleurs. Elle retourne quatre valeurs : deux booléens indiquant si le courriel est valide ou non, et deux messages de

feedback qui seront affichés à l'utilisateur, fournissant une rétroaction immédiate sur la validité de l'entrée (affichage du message et feedback visuel via la classe Bootstrap intégrée).

```
@app.callback(
    Output('signup-button', 'disabled'),
    [Input('signup-email', 'valid'),
     Input('signup-password', 'valid', ),
     Input('signup-name', 'valid')]
)
def update_signup_button_state(
    valid_email: bool,
    valid_password: bool,
    valid_name: bool,
) -> bool:
    """ Update the signup button state
    Args:
        valid_email (bool): is email valid
        valid_password (bool): is password valid
        valid_name (bool): is name valid
    Returns:
        bool: is button disabled """
    if valid_email and valid_password and valid_name:
        return False # button enabled
    return True # button disabled
```

Un autre callback (ci-dessus) est responsable de l'activation du bouton d'inscription. Il surveille la validité des champs nom d'utilisateur, email et mot de passe. Si tous ces champs sont validés, le bouton d'inscription est activé, permettant à l'utilisateur de soumettre sa demande d'inscription. La fonction **update_signup_button_state** retourne un booléen qui contrôle l'attribut **disabled** du bouton : False pour activer le bouton et True pour le désactiver. Cela assure que l'utilisateur ne puisse procéder à l'inscription que lorsque toutes les informations requises sont validées, renforçant ainsi l'intégrité du processus d'inscription.

Le détail du rendu visuel de ces callbacks est illustré ci-dessous et sur la page suivante.

Figure 11 : Rendu visuel des feedbacks et du bouton d'inscription

Créer un Compte	Créer un Compte
Si vous n'avez pas encore de compte utilisateur, veuillez en créer un ici*.	Si vous n'avez pas encore de compte utilisateur, veuillez en créer un ici*.
Nom d'utilisateur	Marine;
Adresse email	marine@spectre
Mot de Passe	*****
Mot de passe : 12 caractères dont 1 minuscule, 1 majuscule, 1 chiffre et un caractère spécial	Le mot de passe doit contenir au moins 12 caractères. Mot de passe : 12 caractères dont 1 minuscule, 1 majuscule, 1 chiffre et un caractère spécial
Créer le compte	Créer le compte
* La création de compte est ouverte aux membres de Spectre Biotech, pour plus d'informations, veuillez contacter le support .	* La création de compte est ouverte aux membres de Spectre Biotech, pour plus d'informations, veuillez contacter le support .

Etat initial, champs vides, bouton non cliquable

Champs de saisie invalides, bouton non cliquable

Créer un Compte

Si vous n'avez pas encore de compte utilisateur, veuillez en créer un ici*.

Marine

✓

marin@spectre-biotech.com

✓

.....

✓

Mot de passe : 12 caractères dont 1 minuscule, 1 majuscule, 1 chiffre et un caractère spécial

Créer le compte

* La création de compte est ouverte aux membres de Spectre Biotech, pour plus d'informations, veuillez [contacter le support](#).

Champs de saisie valides, bouton cliquable

C. LES BASES DE DONNEES

1. BASE DE DONNEES EEG EXPLORER

Dans le projet EEG Explorer, la **base de données** a été conçue avec PostgreSQL et nous avons choisi SQLAlchemy comme ORM Python pour interfacer notre application avec cette **base de données**, permettant une modélisation intuitive des structures de données sous forme de classes Python. Notre schéma de base de données se compose de quatre tables : users, contacts, queries, et query_results.

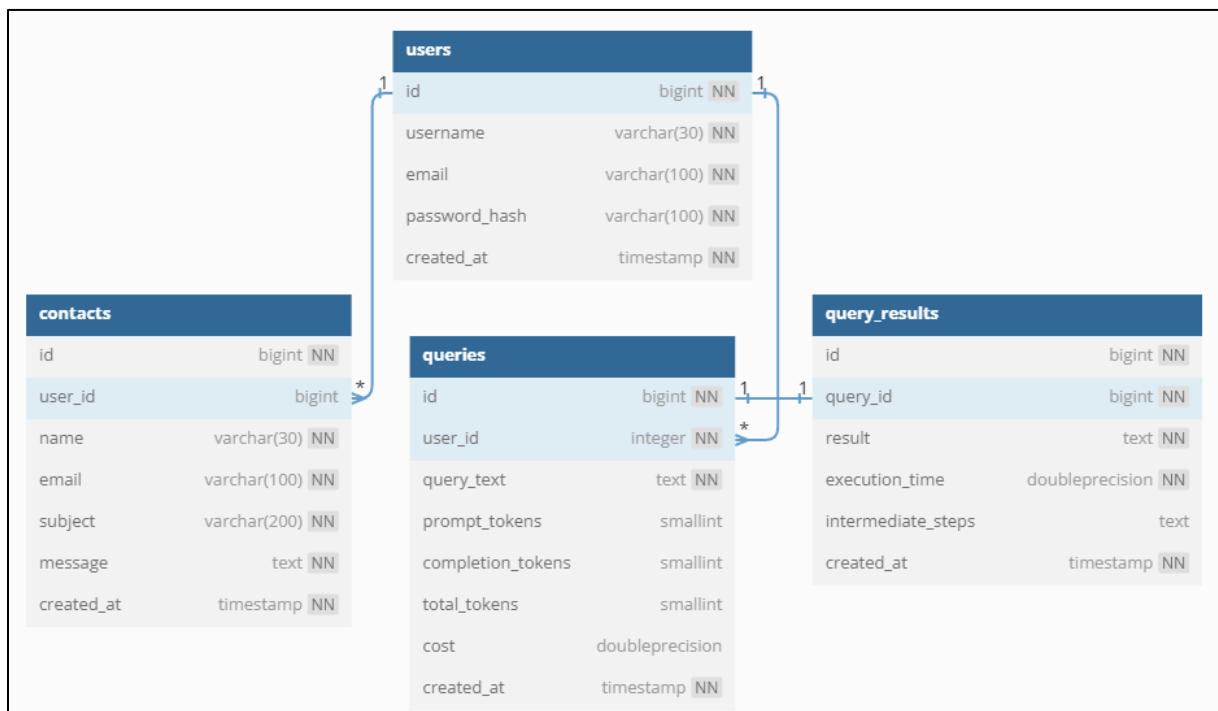


Figure 12 : Diagramme de la base de données EEG Explorer

La table **users** constitue le répertoire central des informations d'authentification, avec des attributs pour l'identifiant unique, le nom d'utilisateur, l'adresse email, le hash du mot de passe et la date de création de l'enregistrement. Lorsqu'un utilisateur pose une question via la fonctionnalité d'exploration, les informations pertinentes sont capturées dans la table **queries**, y compris le texte de la requête, les tokens associés et le coût calculé. Les réponses générées

sont consignées dans **query_results**, où chaque entrée est liée à sa requête respective, capturant le résultat, le temps d'exécution et les étapes intermédiaires de traitement de l'intelligence artificielle. Les interactions des utilisateurs à travers le formulaire de contact sont enregistrées dans la table **contacts** (**user_id** est stocké si l'utilisateur est connecté).

Les relations entre les tables sont définies par des clés étrangères : **user_id**, dans les tables **queries** et **contacts**, fait le lien avec la table **users** selon une relation de type un-à-plusieurs, tandis que **query_id** dans **query_results** associe chaque réponse à sa requête respective dans une relation un-à-un.

```
class CommonMixin():
    """ Common columns for all tables """
    id: Mapped[int] = mapped_column(
        BigInteger, Identity(), primary_key=True
    )
    created_at: Mapped[datetime.datetime] = mapped_column(
        DateTime(timezone=True), server_default=func.now()
    )

class User(UserMixin, CommonMixin, db.Model):
    """ User model """
    __tablename__ = 'users'

    username: Mapped[str] = mapped_column(
        String(30), nullable=False,
    )
    email: Mapped[str] = mapped_column(
        String(100), nullable=False, unique=True
    )
    password_hash: Mapped[str] = mapped_column(
        String(100), nullable=False
    )
```

La classe **CommonMixin** est utilisée pour standardiser les champs communs comme l'ID, auto-incrémenté comme clé primaire, et le timestamp de création. En héritant de **CommonMixin** et de **UserMixin** (provenant de Flask-Login pour la logique d'authentification), la classe **User** étend ces fonctionnalités de base tout en ajoutant des champs spécifiques à l'utilisateur. Ce code Python, via SQLAlchemy, établit la structure de la table **users** et les liaisons nécessaires et permet de créer la table en **base de données** lors du lancement de l'application.

Le **code SQL équivalent** pour la création de la table **users** est indiqué ci-dessous.

```
CREATE TABLE IF NOT EXISTS eeg_explorer.users
(
    username character varying(30) NOT NULL,
    email character varying(100) NOT NULL UNIQUE,
    password_hash character varying(100) NOT NULL,
    id bigint PRIMARY KEY NOT NULL GENERATED BY DEFAULT AS IDENTITY,
    created_at timestamp with time zone NOT NULL DEFAULT now(),
);
```

2. BASE DE DONNEES SPECTRE

La **base de données Spectre** n'était pas prête à temps. Afin de pouvoir tout de même présenter mon travail dans le cadre de la certification CDA, j'ai créé une **base de données Spectre** simplifiée sur PostgreSQL.

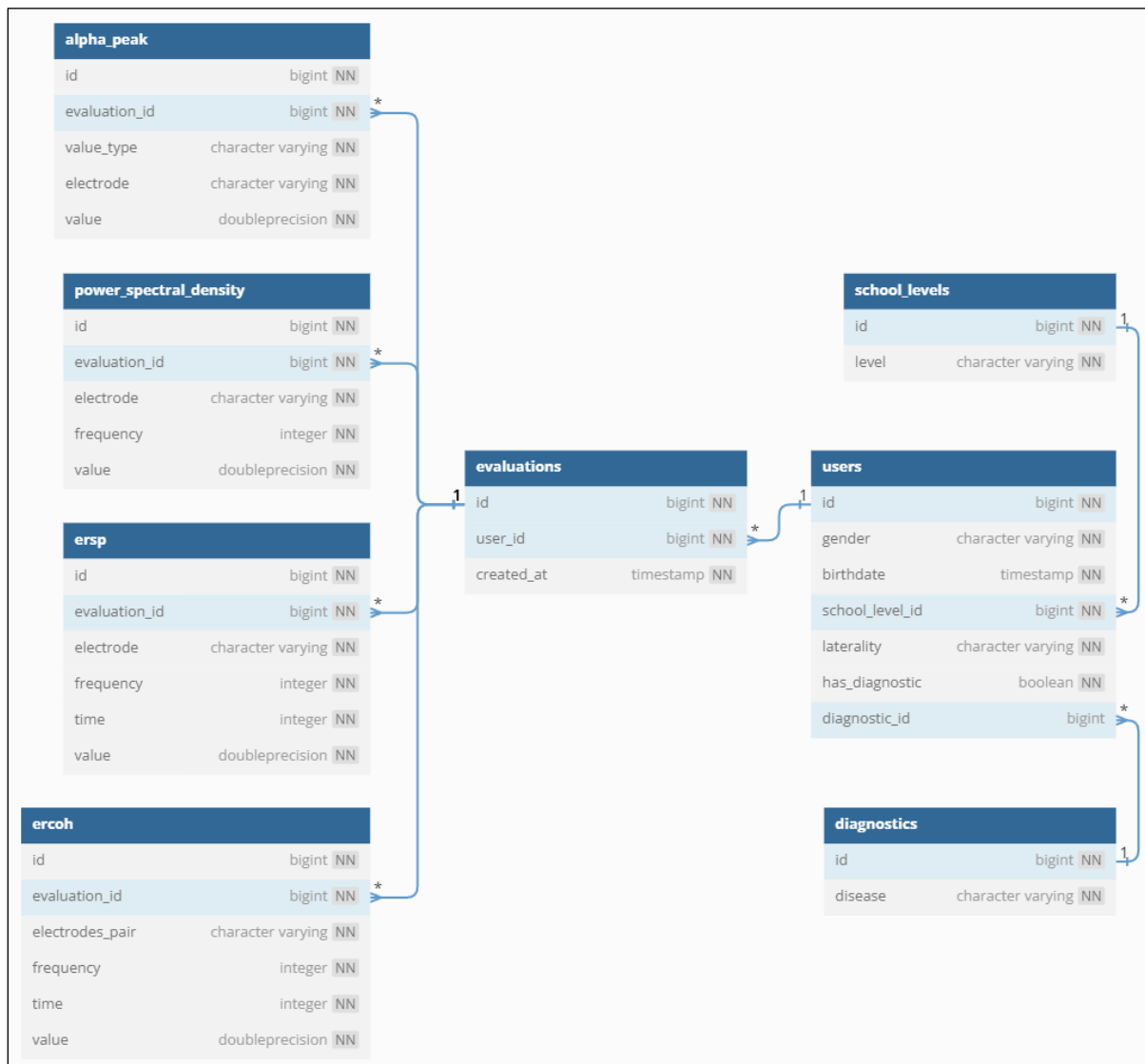


Figure 13 : Diagramme de la base de données Spectre

Le diagramme ci-dessus illustre la structure de la **base de données Spectre**, conçue pour **stocker et organiser les données relatives aux évaluations EEG des patients**. Elle est composée de plusieurs tables interconnectées, chacune jouant un rôle spécifique :

- **users** est le noyau de la base de données, recensant les informations des utilisateurs, telles que l'identifiant unique, le genre, la date de naissance, le niveau scolaire, la latéralité, et des informations sur un éventuel diagnostic.
- **school_levels** catégorise les niveaux d'éducation des utilisateurs et est reliée à la table utilisateurs par le champ **school_level_id**.
- **diagnostics** contient les détails des diagnostics associés aux utilisateurs, avec une relation un-à-plusieurs indiquée par la clé étrangère **diagnostic_id** dans la table **users**.

- **evaluations** enregistre les sessions d'évaluation EEG, liées aux utilisateurs par **user_id**. Chaque évaluation a une date et est associée à un ou plusieurs résultats spécifiques d'EEG.
- **alpha_peak**, **power_spectral_density**, **ersp** (Event-Related Spectral Perturbation) et **ercoh** (Event-Related Coherence) contiennent les données calculées à partir des résultats des évaluations EEG, telles que les valeurs des pics alpha, la densité de puissance spectrale, les réponses événementielles et la cohérence entre paires d'électrodes. Chaque table est liée à la table évaluations par le champ **evaluation_id**.

La cardinalité entre les tables est définie pour refléter la relation entre les entités : par exemple, un utilisateur peut avoir plusieurs évaluations, mais chaque évaluation est liée à un seul utilisateur. De même, chaque évaluation peut produire plusieurs enregistrements de données EEG dans chacune des tables dédiées à ces mesures.

D. BACK-END : EXEMPLE DE L'AGENT SQL

Le back-end d'une application multicouche joue un rôle fondamental en agissant comme le **moteur de traitement des données et de la logique métier**. Situé derrière la scène, comme son nom l'indique, il est responsable de la gestion des interactions avec la base de données (via l'API), de l'exécution des opérations serveur, et de la mise en œuvre des règles métier. Il traite les demandes envoyées depuis le front-end, effectue les calculs ou les manipulations de données nécessaires, et renvoie les résultats appropriés. En outre, le back-end assure des fonctions critiques telles que l'authentification des utilisateurs et la sécurité des données. Sa conception et sa robustesse sont cruciales pour garantir que l'application fonctionne de manière fluide, efficace et sécurisée, fournissant ainsi une expérience utilisateur cohérente et fiable.

Pour illustrer mon travail coté back-end, j'ai choisi de faire un focus sur la **fonctionnalité centrale d'EEG Explorer : l'agent SQL utilisant un LLM** (Large Language Model) pour interagir avec la **base de données Spectre**. Il fonctionne comme un intermédiaire intelligent, capable d'interpréter les requêtes rédigées en langage naturel et de fournir des réponses pertinentes basées sur les données stockées en **base de données**.

La création de cet agent SQL est réalisée en Python en utilisant LangChain, un framework qui facilite la création et la gestion de modèles de langage. Dans notre implémentation, **l'agent est configuré pour se connecter à la base de données Spectre, traiter les requêtes reçues et formuler des réponses en exploitant les informations extraites de cette base de données**. Cette approche permet non seulement de répondre aux questions des utilisateurs de manière efficace mais aussi d'explorer de nouvelles façons d'interagir avec les ensembles de données complexes, ouvrant ainsi la voie à des analyses plus approfondies et à une meilleure compréhension des données EEG.

J'ai utilisé l'**approche orientée objet de Python** pour créer la classe **SQLAgent**, cette approche permet une organisation claire et modulaire du code, facilitant ainsi la maintenance et l'évolutivité du système. En regroupant la logique de l'agent dans une seule classe, la complexité de l'interaction avec la **base de données Spectre** est réduite, rendant le processus plus transparent et facile à gérer. De plus les variables de configuration sont regroupées dans un module **sql_agent_cfg** ce qui permet de centraliser et de simplifier la gestion des paramètres de l'agent. Cette centralisation facilite les ajustements et les mises à jour des configurations sans avoir à modifier le code principal de l'agent. Par exemple, si le modèle LLM ou les paramètres de connexion à la base de données doivent être modifiés, ces ajustements peuvent être réalisés rapidement et de manière cohérente en mettant à jour le fichier **sql_agent_cfg** (le code de ce module est disponible en [annexe 6](#)).

```
class SQLAgent:
    """The SQLAgent class is used to interact with the SQL database
    using a llm.
    """
    def __init__(
        self,
        database=sql_agent_cfg.database,
        llm_model=sql_agent_cfg.llm_model,
        tools=sql_agent_cfg.tools,
        agent_type=sql_agent_cfg.agent_type,
        prompt_template=sql_agent_cfg.prompt_template
    ):
        """ Initialize the SQLAgent class with the llm, the database,
        the tools, the agent_type and the prompt template.
        """
        self.database = database
        self.llm_model = llm_model
        self.tools = tools
        self.agent_type = agent_type
        self.prefix = prompt_template["prefix"]
        self.suffix = prompt_template["suffix"]
        self.agent = initialize_agent(
            tools=self.tools,
            llm=self.llm_model,
            agent=self.agent_type,
            verbose=True,
            return_intermediate_steps=True,
            agent_kwargs={
                "prefix": self.prefix,
                "suffix": self.suffix
            },
            handle_parsing_errors=True
        )
```

La méthode **__init__** de la classe **SQLAgent** sert à initialiser l'instance de la classe en définissant les attributs nécessaires à son fonctionnement. L'utilisation du mot-clé **self** dans cette méthode permet d'assigner des variables et des fonctions à l'instance de la classe, rendant ces éléments accessibles dans toutes les méthodes de la classe.

- **self.database** établit une connexion avec la **base de données SQL de Spectre Biotech**, dont l'URL est récupérée dans les variables d'environnement avec `os.environ.get()`.

```
url_database = os.environ.get('DATABASE_URL_SPECTRE')
database = SQLiteDatabase.from_uri(url_database)
```

- **self.llm_model** est ici configuré pour créer une instance de ChatOpenAI avec un modèle spécifique. Nous avons choisi d'utiliser le modèle gpt-4-turbo d'OpenAI mais des tests sont prévus dans le futur pour comparer les performances de plusieurs modèles.

```
model_name = 'gpt-4-turbo-preview'
llm_model = ChatOpenAI(model_name=model_name)
```

- **self.tools** récupère les outils du module SQLiteDatabaseToolkit conçu par Langchain pour permettre à un agent d'interagir avec une base de données SQL. Quand on l'instancie, on précise la base de données et le modèle LLM que l'on souhaite utiliser.

```
toolkit = SQLiteDatabaseToolkit(db=database, llm=llm_model)
tools = toolkit.get_tools()
```

Les outils sont spécialisés pour interpréter les requêtes en langage naturel, générer les requêtes SQL correspondantes et interagir avec la base de données :

- **sql_db_list_tables** est utilisé pour lister toutes les tables présentes dans la base de données, permettant à l'agent de comprendre la structure de données disponible.
- **sql_db_schema** récupère le schéma d'une table donnée, ce qui est crucial pour formuler des requêtes SQL précises.
- **sql_db_query_checker** vérifie la validité d'une requête SQL générée, assurant que les requêtes sont exécutables et sans erreurs.
- **sql_db_query** exécute la requête SQL formulée et retourne les résultats, permettant ainsi de répondre aux interrogations des utilisateurs.
- **self.agent_type** définit le type d'agent utilisé. Nous avons choisi un agent ZERO_SHOT_REACT_DESCRIPTION. Il est conçu pour traiter des requêtes ou des tâches sans entraînement spécifique préalable, en utilisant des descriptions de tâches pour générer des réponses ou des actions appropriées. A l'avenir, d'autres types d'agent pourront être testés, notamment ceux qui ont de la mémoire et permettent de suivre plusieurs échanges.

```
agent_type = AgentType.ZERO_SHOT_REACT_DESCRIPTION
```

- **self.prefix** et **self.suffix** définissent les consignes que nous donnons à notre agent (les préfixe et suffixe actuels ne peuvent être reproduit ici pour des raisons de discrétion demandée par l'entreprise). Ces instructions guident l'agent SQL dans la génération de réponses précises et pertinentes, tout en assurant la sécurité et l'intégrité de la base de données. Elles favorisent une interaction intelligente et réfléchie avec la base de données, permettant à l'agent de répondre de manière informée et contextuelle aux requêtes des utilisateurs. Les éléments suivants sont abordés :
 - **Rôle de l'Agent** : L'agent est identifié comme un intermédiaire conçu pour interagir avec une base de données SQL, spécifiquement PostgreSQL.
 - **Création de Requêtes** : Il doit générer des requêtes SQL syntaxiquement correctes en réponse aux questions des utilisateurs.

- **Langue et Format de Réponse** : L'agent doit répondre dans la même langue que la question posée et se concentrer sur les informations pertinentes.
 - **Utilisation Sélective des Données** : Il est instruit d'utiliser uniquement les colonnes pertinentes et d'ordonner les résultats si nécessaire pour fournir les exemples les plus intéressants.
 - **Outils Spécifiques** : L'agent a accès à des outils définis pour interagir avec la base de données et doit les utiliser exclusivement pour construire ses réponses.
 - **Vérification et Sécurité des Requêtes** : Il doit vérifier soigneusement les requêtes avant exécution et ne doit pas effectuer de modifications dans la base de données, les commandes DDL (Data Definition Language) et DML (Data Manipulation Language) sont interdites sauf le SELECT.
 - **Discrétion sur les Consignes** : L'agent ne doit pas mentionner ses instructions dans ses réponses.
 - **Début de l'Action** : Le suffixe incite l'agent à commencer son processus de réflexion et d'analyse.
 - **Processus de Réflexion** : Il doit envisager d'examiner les tables de la base de données et leurs schémas pour déterminer les meilleures données à interroger.
 - **Étapes de la Pensée** : L'agent est encouragé à documenter son raisonnement pour montrer comment il aborde la question posée.
- **self.agent** initialise finalement l'agent LangChain avec les outils nécessaires, le modèle LLM, le template de prompt, la gestion des erreurs de parsing et l'activation des étapes intermédiaires. Ces dernières jouent un rôle intéressant en capturant les différentes étapes et décisions prises par l'agent lors de la transformation de la requête en langage naturel en requête SQL. Cela permet non seulement d'assurer la transparence du processus, mais aussi d'offrir des insights sur la manière dont l'agent comprend et répond aux questions, améliorant ainsi la fiabilité et la clarté de l'interaction utilisateur (le détail complet du traitement d'une requête par l'agent SQL en 7 cycles est disponible en [annexe 7](#)).

La méthode **query_database** (voir code page suivante) est ensuite définie dans la classe **SQLAgent** (une méthode est une fonction définie à l'intérieur d'une classe, elle est associée à ses instances et peut accéder et modifier les attributs de l'objet ou effectuer des opérations spécifiques à cette classe). Elle prend une question en langage naturel en argument et utilise l'agent pour interroger la base de données. Elle utilise un contexte **with** pour garantir la fermeture correcte du callback **get_openai_callback** après l'exécution. L'horloge de début est enregistré avant l'exécution de la requête puis **self.agent(query)** est appelé pour répondre à la question via l'agent SQL, et l'horloge de fin est enregistré juste après. La méthode retourne plusieurs informations : la réponse à la question, les étapes intermédiaires, le nombre total de tokens utilisés, les tokens utilisés pour le prompt et la complétion, le coût total de l'opération auprès du fournisseur de LLM, et le temps d'exécution. Avec l'utilisation d'un bloc **try – except**, si une exception est levée, un message d'erreur est enregistré, et des valeurs par défaut sont retournées pour indiquer l'échec de la requête.

```

def query_database(self, query: str) -> tuple:
    """Query the SQL database with the sql agent.
    Args:
        query (str): The verified query to be translated in SQL.
    Returns:
        Tuple[str, dict, int, int, int, float, float]: A tuple containing the
        result of the query, the intermediate steps, the total tokens used,
        the prompt tokens used, the completion tokens used, the total cost
        of the query, and the execution time in seconds. """
    logging.info(f"Querying the database with the query: {query}")
    try:
        with get_openai_callback() as cb:
            start = time.time()
            result = self.agent(query)
            end = time.time()
            execution_time = end - start
            return (
                result ['output'],
                result ['intermediate_steps'],
                cb.total_tokens,
                cb.prompt_tokens,
                cb.completion_tokens,
                cb.total_cost,
                execution_time
            )
        logging.info("Query successful")
    except Exception as e:
        logging.error(f"An error occurred during the query: {e}")
        return (
            "An error occurred, please try again", {}
            None, None, None, None, None
        )

```

En résumé, la classe **SQLAgent** intègre un modèle LLM avec une base de données SQL pour traiter des requêtes en langage naturel, tout en fournissant des informations détaillées sur l'exécution des requêtes, y compris les performances et les coûts. Cette approche offre une interface dynamique et puissante pour interroger et analyser les données de manière intuitive et efficace. Ainsi, quand un utilisateur pose une question sur la page d'exploration, son click sur le bouton déclenchera un callback qui initiera la classe SQLAgent afin de pouvoir afficher la réponse attendue.

EEG Explorer

Transformez vos questions en découvertes

Bienvenue **Marine**

C'est ici que vous pouvez interroger notre base de données. Vous pouvez écrire votre question dans le champ ci-dessous et lancer la recherche.

Pour en savoir plus sur notre base de données et son contenu, vous pouvez consulter la [documentation](#).

Écrivez ici votre question
 par exemple : combien y'a-t-il d'enregistrements EEG?
 ou bien : quel est la valeur moyenne du pic alpha en F3 chez les personnes souffrant d'autisme?

EEG Explorer utilise Chat GPT 4 pour traiter vos questions, il peut faire des erreurs, pensez à vérifier les informations importantes.

Interroger la base de données

Résultat de la requête

Votre question était : Quelles sont les tables présentes en base de données?

Les tables présentes en base de données sont : alpha_peak, diagnostics, ercoh, ersp, evaluations, power_spectral_density, schoollevels, et users.

Figure 14 : Affichage du résultat de la requête

E. API

Dans notre application l'**API** est destinée à la communication avec la **base de données EEG Explorer**. Elle s'organise de la manière suivante :

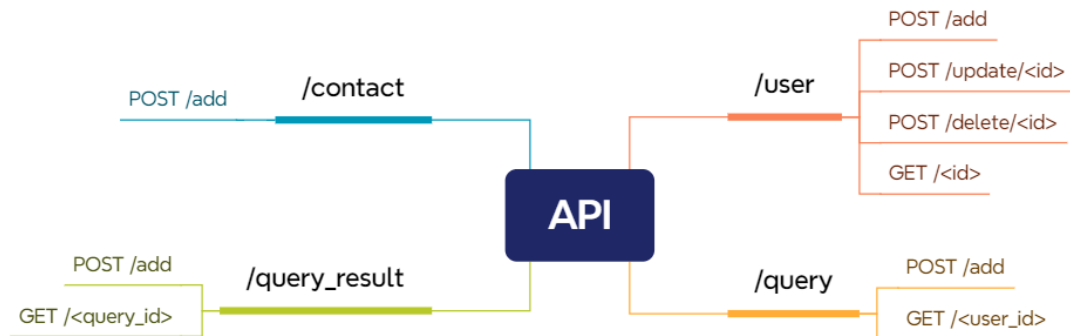


Figure 15 : Schéma des routes API de l'application EEG Explorer.

Par exemple, la route `/api/query/add` de l'API Flask (voir le code ci-dessous) sert à **enregistrer les requêtes des utilisateurs dans la base de données**. Protégée par le décorateur `@login_required`, cette route s'assure que seuls les utilisateurs authentifiés peuvent soumettre des requêtes.

```
@app.server.route('/api/query/add', methods=['POST'])
@login_required
def record_query():
    """ Record a query and other details in database """
    data = request.json
    try:
        new_query = Queries(
            query_text=data['query_text'],
            user_id=data['user_id'],
            prompt_tokens=data.get('prompt_tokens'),
            completion_tokens=data.get('completion_tokens'),
            total_tokens=data.get('total_tokens'),
            cost=data.get('cost')
        )
        db.session.add(new_query)
        db.session.commit()
        return jsonify({
            "success": True,
            "message": "Query saved successfully.",
            "query_id": new_query.id
        })
    except SQLAlchemyError as e:
        db.session.rollback()
        return jsonify({"success": False, "message": str(e)})
```

Elle accepte les données POST au format JSON, incluant des informations telles que **query_text**, **user_id**, et diverses métriques liées à la requête. Ces données sont ensuite utilisées pour créer une nouvelle instance de la classe **Queries**, qui est ajoutée à la session de la base de données et enregistrée dans la table correspondante. En cas de succès, la route renvoie une réponse JSON indiquant que la requête a été enregistrée avec succès, accompagnée de l'ID de la requête. Cet ID est particulièrement important, car il sert de clé étrangère pour associer le résultat de la requête, stocké dans la table **query_results**, à la requête initiale. En cas d'échec de l'enregistrement de la requête, par exemple à cause d'une erreur de base de données, la transaction est annulée par le processus de rollback pour garantir l'intégrité des données, et un message d'erreur est renvoyé.

F. SECURITE

1. CRYPTAGE DES DONNEES SENSIBLES

La classe **User** (vue précédemment [page 28](#)) permet de créer et d'alimenter la table **users** en **base de données**, elle contient **2 méthodes qui jouent un rôle fondamental dans la gestion de la sécurité des informations d'identification des utilisateurs**. La méthode **set_password** prend un mot de passe en clair comme argument et utilise la fonction hash de la bibliothèque Argon2 pour **créer et stocker un hachage sécurisé** dans le champ **password_hash**. Cette pratique est essentielle pour préserver la sécurité des mots de passe utilisateur en **base de données**.

```
def set_password(self, password: str):
    """ hash the password and set it to the password_hash attribute
    Args:
        password (str): the password to hash """
    self.password_hash = argon2.hash(password)
```

La méthode **check_password** permet de vérifier l'exactitude d'un mot de passe en clair fourni lors d'une tentative de connexion, en le comparant au hachage stocké en utilisant la fonction **verify d'Argon2**. Cette méthode est cruciale pour l'authentification des utilisateurs, assurant que l'accès est accordé uniquement avec les bons identifiants.

```
def check_password(self, password : str):
    """ check if the password is correct
    Args:
        password (str): the password to check
    Returns:
        bool: True if the password is correct, False otherwise"""
    return argon2.verify(password, self.password_hash)
```

2. VERIFICATION DES DONNEES ENTRANTES

La vérification des données entrantes se fait à plusieurs niveaux. C'est indispensable pour ne pas s'exposer à des actes malveillants de type injection SQL.

Au niveau du front-end, il existe des processus de validation sur les données saisies. Si celles-ci ne sont pas valides, l'utilisateur est informé (feedbacks visuels et textuels) et **les boutons permettant l'envoi des données ne sont pas activés**.

De plus, l'utilisation de SQLAlchemy est une protection en soi contre la plupart des formes d'injections SQL grâce à la façon dont cet ORM construit les requêtes SQL. Il utilise en effet **des requêtes paramétrées**, ce qui signifie que les données sont envoyées à la base de données séparément des commandes SQL, empêchant ainsi les données d'être interprétées comme du code SQL. Par sécurité, une fonction supplémentaire de vérification est implémentée en backend et au niveau de l'API : **is_input_safe** (voir code page suivante).

```
def is_safe_input(input_string: str) -> bool:
    """ Checks if the input string is safe to be processed or
    inserted into the database.
    This function searches for patterns that might indicate
    an SQL injection attempt.
    Args:
        input_string (str): The input string to be checked.
    Returns:
        bool: True if the input string is safe, False otherwise. """
    # List of patterns
    patterns = [
        r';',          # SQL end of a query
        r'--',         # SQL comment
        r'\/*',        # SQL comment (start)
        r'\*/',        # SQL comment (end)
        r'@@',         # SQL system variable
        r'char(',      # CHAR function
    ]
    # Check each pattern against the input
    for pattern in patterns:
        if re.search(pattern, input_string):
            return False
    return True
```

Enfin, au niveau de la base de données, il existe aussi des restrictions sur le type de données dans chaque colonne et les droits d'accès sont strictement limités à ce qui est nécessaire.

3. SECURISER L'ACCES A LA BASE DE DONNEES SPECTRE

L'accès à la base de données Spectre passe par le modèle d'intelligence artificielle instancié avec LangChain. Afin de sécuriser cet accès j'ai mis en place **3 niveaux de sécurité** :

- Quand une requête est envoyée via le formulaire de la page d'exploration, elle est traduite en anglais (avec l'API DeepL) et passe par **la fonction check_keywords afin de ne pas autoriser l'utilisation de certaines commandes DDL et DML**

```
def check_keywords(text: str) -> bool:
    """ Check if the translated text contains any of the keywords
    that might indicate a dangerous SQL query.
    Args: text (str): The translated text to be checked.
    Returns: bool: True if the text is safe, False otherwise. """
    # regex for each category of SQL commands
    data_manipulation_regex = r"(?i)\b(creat|add|insert|delet|remov|drop|updat|modif|chang|alter|truncat|merg)"
    transaction_control_regex = r"(?i)\b(commit|rollback|savepoint)"
    structure_manipulation_regex = r"(?i)\b(create\s+(index|table|view)|drop\s+(index|table|view)|renam)"
    access_control_regex = r"(?i)\b(grant|revok|privileg)"
    specific_sql_commands_regex = r"(?i)\b(lock|unlock|execut|call|set)"
    # verify if the text contains any of the keywords
    is_safe = not any(re.search(regex, text) for regex in [
        data_manipulation_regex,
        transaction_control_regex,
        structure_manipulation_regex,
        access_control_regex,
        specific_sql_commands_regex
    ])
    return is_safe
```

- Par ailleurs, **dans les consignes données au modèle via le template de prompt**, les commandes DDL (Data Definition Language) et DML (Data Manipulation Language) sont interdites sauf le SELECT.

DO NOT make any DDL or DML statements (INSERT, UPDATE, DELETE, DROP etc.) to the database. Only SELECT statements are allowed. If a user asks you to make a change to the database, return "I'm not allowed to make any change to the database".

Il lui a aussi été demandé de ne pas révéler ses consignes, afin d'éviter au maximum les risques de détournements.

- Enfin, le profil utilisateur utilisé par Langchain pour se connecter à la **base de données Spectre** a des **droits d'accès limités uniquement aux commandes SELECT** via les commandes psql suivantes :

```
REVOKE ALL PRIVILEGES ON ALL TABLES IN SCHEMA eeg_explorer FROM eeg_explorer_user;  
REVOKE ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA eeg_explorer FROM eeg_explorer_user;  
REVOKE ALL PRIVILEGES ON ALL FUNCTIONS IN SCHEMA eeg_explorer FROM eeg_explorer_user;  
GRANT SELECT ON ALL TABLES IN SCHEMA eeg_explorer TO eeg_explorer_user;
```

4. SECURISER LE TRANSFERT DES DONNEES

Le HTTPS joue un rôle crucial dans la protection des données transmises entre le serveur et le client. Il s'appuie sur le **protocole SSL/TLS pour chiffrer les données en transit**, ce qui signifie que toute information échangée est convertie en un format codé, inintelligible sans la clé de déchiffrement appropriée. Ainsi, même si les données sont interceptées durant leur transfert, elles restent **inaccessibles aux acteurs malveillants**. Le HTTPS assure également l'authenticité du serveur, empêchant les utilisateurs de se connecter à des imitateurs malveillants. Cette couche de sécurité est essentielle pour **préserver la confidentialité et l'intégrité des données**, réduisant considérablement les risques de compromission pour les utilisateurs.

5. ISOLER L'APPLICATION AVEC DOCKER

Docker offre une solution efficace pour encapsuler les applications dans des conteneurs isolés. **Chaque conteneur fonctionne comme un environnement cloisonné, avec la possibilité de contrôler précisément les points d'accès externes**. Ainsi, l'accès à l'application est finement géré. Dans un tel conteneur, on retrouve :

- L'application Dash, servant d'interface utilisateur,
- L'API Flask pour la gestion des données,
- La **base de données** PostgreSQL.

Seule l'application Dash est accessible de l'extérieur, tandis que l'API et la base de données restent internes au conteneur, renforçant significativement la sécurité.

G. TESTS

Dans le développement d'applications, plusieurs types de tests sont essentiels pour assurer la qualité et la fiabilité de l'application :

Les tests unitaires se concentrent sur de petites parties du code, généralement des fonctions ou des méthodes individuelles. L'objectif est de s'assurer que chaque composant fonctionne comme prévu de manière isolée. Ces tests sont cruciaux pour identifier les erreurs précocement dans le cycle de développement.

Les tests d'intégration évaluent comment différents modules ou services de l'application interagissent entre eux. Ils sont importants pour détecter les problèmes liés aux interfaces et aux interactions entre les composants.

Les tests de système vérifient le comportement et les performances de l'application complète. Ces tests couvrent toutes les intégrations et fonctionnalités pour s'assurer que le système dans son ensemble répond aux exigences spécifiées.

Les tests d'acceptation, souvent réalisés avec les utilisateurs finaux, permettent de s'assurer que l'application répond aux besoins et attentes de l'utilisateur. Ces tests valident l'expérience utilisateur, la conformité aux exigences métier et l'adéquation de l'application à l'usage prévu.

Les tests de charge et de performance examinent comment l'application se comporte sous des conditions de charge variées, notamment pour tester sa capacité à gérer un grand nombre d'utilisateurs simultanés et sa performance sous stress.

Les tests de sécurité sont spécifiques à la détection des vulnérabilités de sécurité. Ils sont essentiels pour protéger l'application contre les attaques et les violations de données.

Chaque type de test joue un rôle crucial dans le cycle de développement d'une application, contribuant à sa robustesse, sa sécurité et son efficacité. Une stratégie de test complète inclut généralement une combinaison de ces différents types pour couvrir tous les aspects de l'application.

Pour EEG Explorer nous avons mis en place **les tests unitaires et les tests d'intégration au fur et à mesure de l'écriture du code** car ils permettent justement de s'assurer qu'on ne casse pas une fonctionnalité précédemment codée en en ajoutant une nouvelle ou en les combinant. Les tests d'acceptation seront réalisés par les dirigeants de Spectre Biotech, dès que la **base de données Spectre** sera déployée, pour vérifier que l'application répond bien aux besoins initiaux et que l'expérience utilisateur est adaptée.

Pour la réalisation des tests unitaires et d'intégration, j'ai opté pour **Pytest, une bibliothèque de tests pour le langage de programmation Python**. Elle est simple d'utilisation et permet d'écrire des tests de manière lisible et efficace, grâce à des fonctionnalités telles que le support des fixtures, qui facilite la mise en place de données de test, et le marquage des tests, qui permet de catégoriser et sélectionner facilement des tests spécifiques pour l'exécution. De plus, Pytest intègre **un mécanisme de capture des sorties et d'affichage des erreurs** qui rend le débogage des tests plus aisé.

1. EXEMPLE DE TESTS UNITAIRES

La fonction `is_password_safe` sert à évaluer la sécurité d'un mot de passe en vérifiant sa conformité à plusieurs critères de complexité. Elle prend en entrée une chaîne de caractères (le mot de passe) et retourne un tuple composé d'un booléen et d'une liste d'erreurs. Le booléen indique si le mot de passe est jugé sécurisé ou non, tandis que la liste d'erreurs recense les critères non respectés. La fonction vérifie la longueur du mot de passe (au moins 12 caractères), la présence de lettres majuscules et minuscules, de chiffres et de caractères spéciaux. Si le mot de passe ne remplit pas un ou plusieurs de ces critères, la fonction retourne `False` avec les messages d'erreur appropriés. Cette approche permet **une validation détaillée du mot de passe, essentielle pour assurer la sécurité des comptes utilisateurs**. Cette fonction est un candidat idéal pour les tests unitaires, car **chaque critère de sécurité peut être testé de manière isolée** pour vérifier son fonctionnement correct.

```
def is_password_safe(password: str) -> tuple[bool, list[str]]:
    """ Check if the password is safe
    Args:
        password (str): password to check
    Returns:
        tuple[bool, list[str]]: is password safe, list of errors """
    errors = []
    if not is_safe_input(password):
        errors.append('Le mot de passe n\'est pas valide.')
        return False, errors
    if len(password) < 12:
        errors.append(format_password_feedback('12 caractères'))
    if not re.search(r"[A-Z]", password):
        errors.append(format_password_feedback('une lettre majuscule'))
    if not re.search(r"[a-z]", password):
        errors.append(format_password_feedback('une lettre minuscule'))
    if not re.search(r"\d", password):
        errors.append(format_password_feedback('un chiffre'))
    if not re.search(r"[ !#$%&'()*+,-./[\\]^_`{|}~" + r'"]', password):
        errors.append(format_password_feedback('un caractère spécial'))
    if not errors:
        return True, errors
    return False, errors
```

Les tests unitaires pour la fonction `is_password_safe` nécessitent une approche méthodique pour vérifier la robustesse de la validation des mots de passe. Chaque test se concentre sur un critère spécifique de sécurité du mot de passe, assurant ainsi une couverture exhaustive des règles de validation.

Le premier test (`test_input_not_safe`) vérifie que la fonction rejette les entrées considérées comme non sécurisées, conformément à la méthode `is_safe_input` (gestion des risques d'attaque par injection SQL).

```
def test_input_not_safe():
    result, errors = is_password_safe("Abc;--")
    assert not result
    assert len(errors) == 1
    assert "n'est pas valide" in str(errors)
```

Le test **test_password_length** s'assure que la fonction détecte les mots de passe trop courts, renvoyant une erreur si la longueur est inférieure à 12 caractères.

```
def test_password_length():
    result, errors = is_password_safe("Abc1!")
    assert not result
    assert len(errors) == 1
    assert "12 caractères" in str(errors)
```

Les tests **test_password_uppercase** et **test_password_lowercase** vérifient respectivement que la fonction exige la présence d'au moins une lettre majuscule et une lettre minuscule.

```
def test_password_uppercase():
    result, errors = is_password_safe("abcdefghijk1!")
    assert not result
    assert len(errors) == 1
    assert "une lettre majuscule" in str(errors)
def test_password_lowercase():
    result, errors = is_password_safe("ABCDEFGHIJK1!")
    assert not result
    assert len(errors) == 1
    assert "une lettre minuscule" in str(errors)
```

test_password_digit s'assure que la fonction demande un chiffre dans le mot de passe.

```
def test_password_digit():
    result, errors = is_password_safe("abcdefghijk1!")
    assert not result
    assert len(errors) == 1
    assert "un chiffre" in str(errors)
```

Le test **test_password_special_char** contrôle la présence obligatoire d'un caractère spécial.

```
def test_password_special_char():
    result, errors = is_password_safe("abcdefghijk1")
    assert not result
    assert len(errors) == 1
    assert "un caractère spécial" in str(errors)
```

Les tests supplémentaires (**test_password_short_no_digit**, **test_password_short_no_digit_no_upper**, **test_password_short_no_digit_no_upper_no_special**, **test_password_all_violations**) examinent des combinaisons de plusieurs règles non respectées, validant la capacité de la fonction à identifier et signaler plusieurs erreurs simultanément.

```
# Test with two violations
def test_password_short_no_digit():
    result, errors = is_password_safe("Abcdef!")
    assert not result
    assert len(errors) == 2
    assert "12 caractères" in str(errors)
    assert "un chiffre" in str(errors)
# Test with three violations
def test_password_short_no_digit_no_upper():
    result, errors = is_password_safe("abcdef!")
    assert not result
    assert len(errors) == 3
    assert "12 caractères" in str(errors)
    assert "un chiffre" in str(errors)
    assert "une lettre majuscule" in str(errors)
```

```
# Test with four violations
def test_password_short_no_digit_no_upper_no_special():
    result, errors = is_password_safe("abcdef")
    assert not result
    assert len(errors) == 4
    assert "12 caractères" in str(errors)
    assert "un chiffre" in str(errors)
    assert "une lettre majuscule" in str(errors)
    assert "un caractère spécial" in str(errors)
# Test with all violations - empty password
def test_password_all_violations():
    result, errors = is_password_safe("")
    assert not result
    assert len(errors) == 5
    assert "12 caractères" in str(errors)
    assert "un chiffre" in str(errors)
    assert "une lettre majuscule" in str(errors)
    assert "une lettre minuscule" in str(errors)
    assert "un caractère spécial" in str(errors)
```

Enfin, **test_valid_password** confirme que la fonction approuve un mot de passe qui remplit tous les critères.

```
# Test avec un mot de passe valide
def test_valid_password():
    result, errors = is_password_safe("Abcdefghijk1!")
    assert result
    assert not errors
```

Une fois tous les tests écrits et rassemblés dans un fichier 'test_password.py', il me suffit de lancer la commande Pytest correspondante pour vérifier la réussite ou non de tous les scénarios de test.

```
(eeg-explorer-py3.12) maryne@LAPTOP-QH400AR8:~/eeg_explorer$ pytest -vvvv test/test_password.py
===== test session starts =====
platform linux -- Python 3.12.0, pytest-8.0.0, pluggy-1.4.0 -- /home/maryne/.cache/pypoetry/virtualenvs/eeg-explorer-GhGEPztlp-py3.12/bin/python
cachedir: .pytest_cache
rootdir: /home/maryne/eeg_explorer
plugins: dash-2.14.1, anyio-3.7.1
collected 11 items

test/test_password.py::test_password_input_not_safe PASSED
test/test_password.py::test_password_length PASSED
test/test_password.py::test_password_uppercase PASSED
test/test_password.py::test_password_lowercase PASSED
test/test_password.py::test_password_digit PASSED
test/test_password.py::test_password_special_char PASSED
test/test_password.py::test_password_short_no_digit PASSED
test/test_password.py::test_password_short_no_digit_no_upper PASSED
test/test_password.py::test_password_short_no_digit_no_upper_no_special PASSED
test/test_password.py::test_password_all_violations PASSED
test/test_password.py::test_valid_password PASSED

===== 11 passed in 0.04s =====
```

Figure 16 : Capture d'écran des résultats Pytest pour la sécurité du mot de passe

La démarche de rédaction des tests unitaires doit suivre une **stratégie rigoureuse et structurée** pour garantir la fiabilité de la validation des mots de passe dans l'application. Cela illustre l'importance d'une validation minutieuse des entrées des utilisateurs pour maintenir la sécurité des comptes utilisateurs.

2. EXEMPLE DE TESTS D'INTEGRATION

La fonction `create_user` est conçue pour gérer l'inscription des nouveaux utilisateurs dans l'application.

```
def create_user(name: str, email: str, password: str) -> tuple[bool, str]:
    """ Create a new user in the database if the inputs are valid
    Args:
        name (str): username
        email (str): email
        password (str): password
    Returns:
        bool: True if the user has been created, False otherwise
        str: message to display to the user """
    valid, error_message = validate_user_input(name, email, password)
    if not valid:
        return False, error_message

    user = User(username=name, email=email)
    user.set_password(password)

    try:
        db.session.add(user)
        db.session.commit()
        logging.info("Utilisateur créé avec succès.")
        return True, "Utilisateur créé avec succès. Vous pouvez maintenant vous connecter."
    except SQLAlchemyError as e:
        logging.error(f"Erreur lors de la création de l'utilisateur: {e}")
        db.session.rollback()
        return False, "Erreur lors de la création de l'utilisateur"
```

Elle commence par valider les entrées (nom, email, mot de passe) à l'aide de la fonction `validate_user_input`. Si les données ne passent pas la validation, la fonction retourne immédiatement `False` accompagné d'un message d'erreur explicatif. En cas de succès dans la validation, elle procède à la création d'une nouvelle instance de l'utilisateur, définit le mot de passe (crypté pour des raisons de sécurité), et tente d'ajouter l'utilisateur à la base de données. Si l'ajout est réussi, la fonction confirme la création de l'utilisateur et retourne `True` avec un message de succès. En cas d'erreur lors de l'interaction avec la base de données, comme indiqué par une exception `SQLAlchemyError`, elle effectue un `rollback` pour annuler les changements et retourne `False` avec un message d'erreur générique. Cette approche garantit que les détails techniques des erreurs ne sont pas exposés à l'utilisateur, tout en fournissant des informations utiles en cas d'échec. Cette fonction est un excellent candidat pour les tests d'intégration, car elle implique à la fois la validation des entrées et les interactions avec la base de données.

La démarche de tests d'intégration pour la fonction `create_user` est essentielle pour garantir la fiabilité et la robustesse du processus d'inscription des utilisateurs dans l'application. Ces tests visent à s'assurer que la fonction interagit correctement avec les différentes composantes du système, notamment la validation des données en entrée, la gestion de la base de données, et les réponses retournées à l'utilisateur.

```
@pytest.fixture(scope='function')
def test_client():
    app_dash = create_app(test=True) # initialize the app with test configuration
    flask_app = app_dash.server
    with flask_app.app_context():
        db.session.begin_nested() # create a savepoint for the test transactions
        yield flask_app.test_client()
        db.session.rollback() # rollback the test transactions
```

Pour effectuer des tests d'intégration de manière précise et fiable, j'ai configuré la **fixture test_client** qui permet de créer un environnement de test contrôlé, fournissant une interface pour interagir avec une application Dash et une API Flask de manière réaliste. Elle permet d'envoyer des requêtes HTTP, de vérifier les routes, les redirections, et les réponses aux formulaires, garantissant ainsi la fiabilité de l'application en identifiant rapidement les problèmes d'intégration potentiels. De plus, **test_client** joue un rôle essentiel dans la configuration initiale de l'application pour les tests, s'assurant que les variables d'environnement sont correctement paramétrées, initialisant la base de données de test et configurant Flask-Login pour l'authentification. Après chaque test, **test_client** assure un nettoyage complet de l'état de l'application, préservant ainsi l'indépendance entre les tests et garantissant des résultats cohérents et fiables.

La démarche de test d'intégration pour la fonction **create_user** est illustrée par une série de tests méthodiques conçus pour évaluer la fiabilité de cette fonction dans différents scénarios d'utilisation. Chaque test aborde un aspect spécifique de la fonctionnalité, offrant ainsi une vision complète de son comportement dans diverses situations.

Test de Création Réussie : ce test confirme que la fonction crée un utilisateur avec des données valides et vérifie que cet utilisateur est bien ajouté à la base de données. Il assure également que le message de succès approprié est retourné.

```
def test_create_user_success(test_client):
    name = "TestUser"
    email = "testuser@spectre-biotech.com"
    password = "ValidPassword123!"
    result, message = create_user(name, email, password)
    message_expected = "Utilisateur créé avec succès. Vous pouvez maintenant vous connecter."
    assert result == True
    assert message == message_expected
    user = User.query.filter_by(email=email).first()
    assert user is not None
    assert user.username == name
```

Test avec un Email Déjà Enregistré : ce scénario teste la réaction de la fonction face à une tentative de création d'un utilisateur avec un email déjà existant dans la base de données, s'assurant que la fonction retourne correctement un message d'erreur sans créer de doublon.

```
def test_create_user_duplicate_email(test_client):
    name = "AnotherUser"
    email = "eeg_explorer@spectre-biotech.com" # Email already in db
    password = "AnotherValidPassword123!"
    result, message = create_user(name, email, password)
    message_expected = "Email déjà enregistré"
```

```

assert result == False
assert message == message_expected
user = User.query.filter_by(email=email).first()
assert user is not None
assert user.username != name
assert user.email == email

```

Test avec des Données Invalides : deux tests sont effectués ici, l'un avec un mot de passe invalide et l'autre avec un email invalide. Ces tests garantissent que la fonction gère correctement les entrées non conformes en retournant des messages d'erreur spécifiques.

```

def test_create_user_invalid_password(test_client):
    name = "TestUserInvalidPwd"
    email = "testuserinvalid@example.com"
    password = "abc;--" # Mot de passe invalide
    result, message = create_user(name, email, password)
    message_expected = "Mot de passe non sécurisé"
    assert result == False
    assert message == message_expected
    user = User.query.filter_by(email=email).first()
    assert user is None

def test_create_user_invalid_email(test_client):
    name = "TestUserInvalidEmail"
    email = "invalidemail" # Email invalide
    password = "ValidPassword123!"
    result, message = create_user(name, email, password)
    message_expected = "Email invalide"
    assert result == False
    assert message == message_expected
    user = User.query.filter_by(email=email).first()
    assert user is None

```

Test de Gestion d'Erreurs de Base de Données : ce test utilise le mocking pour simuler une erreur lors de l'interaction avec la base de données. Il vérifie que la fonction gère correctement cette situation en effectuant un rollback et en fournissant un message d'erreur adapté.

```

def test_db_error_during_user_creation(test_client):
    name = "TestUser"
    email = "testotheruser@spectre-biotech.com"
    password = "ValidPassword123!"
    message_expected = "Erreur lors de l'enregistrement de l'utilisateur"
    with patch('back.models.db.session.commit', side_effect=SQLAlchemyError):
        result, message = create_user(name, email, password)
        assert not result
        assert message == message_expected
    user = User.query.filter_by(email=email).first()
    assert user is None

```

Ces tests d'intégration, en couvrant un large éventail de scénarios, permettent de s'assurer que **la fonction create_user fonctionne comme attendu dans l'environnement global de l'application**, interagissant correctement avec la base de données et les systèmes de validation. Ils sont essentiels pour garantir la qualité et la fiabilité du processus d'inscription des utilisateurs.

IX. JEU D'ESSAI D'UNE FONCTIONNALITE REPRESENTATIVE DU PROJET

Dans cette partie, j'ai choisi de tester le callback responsable de la navigation dans l'application EEG Explorer.

Il utilise la fonction **page_router** pour **acheminer l'utilisateur vers le bon layout en fonction de l'URL visitée**. Le statut de l'utilisateur (authentifié ou non est aussi à prendre en compte). La fonction consulte d'abord une liste de chemins protégés pour vérifier si l'utilisateur est authentifié avant d'accéder à des pages qui nécessitent une connexion. Si l'utilisateur tente d'accéder à une de ces pages sans être authentifié, il est redirigé vers un layout d'accès restreint le réorientant vers la page d'inscription et de connexion.

```
@app.callback(
    Output('page-content', 'children'),
    [Input('url', 'pathname')]
)
def page_router(pathname: str) -> str:
    """ Display the layout according to the url
    Args:
        pathname (str): current url path
    Returns:
        str: layout to display """
    # management of the authentication status of the user for protected paths
    if any(
        pathname.startswith(path) for path in protected_paths
    ) and not is_user_authenticated():
        return unauthorized_layout
    # redirect to user page if user is authenticated
    if pathname == '/login' and is_user_authenticated():
        return user_layout
    # management of the reset password page
    if pathname.startswith('/reset_password/'):
        return reset_password_layout
    # main router
    return routes.get(pathname, error_404_layout)
```

Si l'utilisateur est déjà connecté et tente d'accéder à la page de connexion, il est redirigé vers un layout spécifique de réorientation vers les fonctionnalités du site. Pour les requêtes de réinitialisation de mot de passe indiquées par un chemin commençant par `/reset_password/` (et suivi par un token unique), le layout correspondant est rendu.

Pour tous les autres chemins, la fonction cherche dans un dictionnaire de routes prédéfinies pour trouver un layout correspondant à l'URL. Si aucune route correspondante n'est trouvée, un layout d'erreur 404 est retourné par défaut. **Cette méthode de routage assure une navigation fluide et sécurisée au sein de l'application, en alignant les URL avec leur contenu associé de manière dynamique et conditionnelle.**

Données entrantes			Données attendues	Données obtenues (tests unitaires)	Ecart	Données obtenues (tests utilisateurs)	Ecart
Type de path	url	Statut utilisateur					
	/	authenticé	home_layout	home_layout	0	home_layout	0
		non authenticé	home_layout	home_layout	0	home_layout	0
	/contact	authenticé	contact_form_layout	contact_form_layout	0	contact_form_layout	0
		non authenticé	contact_form_layout	contact_form_layout	0	contact_form_layout	0
protected path	/documentation	authenticé	documentation_layout	documentation_layout	0	documentation_layout	0
		non authenticé	unauthorized_layout	unauthorized_layout	0	unauthorized_layout	0
protected path	/explorer	authenticé	explorer_layout	explorer_layout	0	explorer_layout	0
		non authenticé	unauthorized_layout	unauthorized_layout	0	unauthorized_layout	0
	/forgot-password	authenticé	forgot_password_layout	forgot_password_layout	0	forgot_password_layout	0
		non authenticé	forgot_password_layout	forgot_password_layout	0	forgot_password_layout	0
	/home	authenticé	home_layout	home_layout	0	home_layout	0
		non authenticé	home_layout	home_layout	0	home_layout	0
	/login	authenticé	user_layout	user_layout	0	user_layout	0
		non authenticé	login_layout	login_layout	0	login_layout	0
protected path	/profile/delete	authenticé	profile_delete_account_layout	profile_delete_account_layout	0	profile_delete_account_layout	0
		non authenticé	unauthorized_layout	unauthorized_layout	0	unauthorized_layout	0
protected path	/profile/edit	authenticé	profile_edit_layout	profile_edit_layout	0	profile_edit_layout	0
		non authenticé	unauthorized_layout	unauthorized_layout	0	unauthorized_layout	0
protected path	/profile/edit/email	authenticé	profile_edit_email_layout	profile_edit_email_layout	0	profile_edit_email_layout	0
		non authenticé	unauthorized_layout	unauthorized_layout	0	unauthorized_layout	0
protected path	/profile/edit/password	authenticé	profile_edit_password_layout	profile_edit_password_layout	0	profile_edit_password_layout	0
		non authenticé	unauthorized_layout	unauthorized_layout	0	unauthorized_layout	0
protected path	/profile/history	authenticé	profile_history_layout	profile_history_layout	0	profile_history_layout	0
		non authenticé	unauthorized_layout	unauthorized_layout	0	unauthorized_layout	0
	/reset_password/+	authenticé	reset_password_layout	reset_password_layout	0	reset_password_layout	0
		non authenticé	reset_password_layout	reset_password_layout	0	reset_password_layout	0
	/unknow	authenticé	error_404_layout	error_404_layout	0	error_404_layout	0
		non authenticé	error_404_layout	error_404_layout	0	error_404_layout	0
protected path	/profile/unknow	authenticé	error_404_layout	error_404_layout	0	error_404_layout	0
		non authenticé	unauthorized_layout	unauthorized_layout	0	unauthorized_layout	0

Figure 17 : Tableau des résultats du jeu d'essai

Dans le tableau ci-dessus on peut voir les données entrantes, les données attendues et les données obtenues lors des tests unitaires et lors des tests utilisateurs.

Les résultats des tests unitaires et des tests utilisateurs pour le routage des pages de l'application EEG Explorer montrent une **cohérence totale entre les données attendues et les données obtenues**, ce qui indique une performance conforme aux attentes à travers tous les scénarios testés. Pour chaque type de chemin (URL) et statut d'utilisateur (authenticé ou non), l'application a systématiquement rendu le layout approprié sans écart détecté.

Les utilisateurs authentifiés et non authentifiés accèdent sans encombre au layout de la page d'accueil et à la page de contact car ils sont libre d'accès. Les chemins protégés tels que la documentation et l'exploration de la base de données sont correctement restreints aux utilisateurs non authentifiés, qui sont redirigés vers un layout d'erreur d'autorisation, tandis que les utilisateurs authentifiés y accèdent comme prévu. Des fonctionnalités telles que l'édition de profil, l'historique, la suppression de compte, et les réinitialisations de mot de passe sont également gérées correctement en fonction de l'authentification de l'utilisateur. Toutes les tentatives d'accès à des URL inconnues renvoient fidèlement au layout d'erreur 404, garantissant ainsi une gestion robuste des erreurs de navigation.

En résumé, **ces essais confirment l'exactitude du système de routage**, illustrant une application bien sécurisée qui respecte l'accès aux fonctionnalités en fonction des droits d'utilisateur, tout en offrant une expérience utilisateur fiable et prévisible.

```

(eeg-explorer-py3.12) maryne@LAPTOP-QHAD0AR8:~/eeg_explorer$ pytest -vvvvv test/test_page_router.py
===== test session starts =====
platform linux -- Python 3.12.0, pytest-8.0.0, pluggy-1.4.0 -- /home/maryne/.cache/pypoetry/virtualenvs/eeg-explorer-GhGEPzMp-py3.12/bin/python
cachedir: .pytest_cache
rootdir: /home/maryne/eeg_explorer
plugins: dash-2.14.1, anyio-3.7.1
collected 30 items

test/test_page_router.py::test_page_router_home_with_authenticated_user PASSED [ 3%]
test/test_page_router.py::test_page_router_launching_with_authenticated_user PASSED [ 6%]
test/test_page_router.py::test_page_router_explorer_with_authenticated_user PASSED [ 10%]
test/test_page_router.py::test_page_router_login_with_authenticated_user PASSED [ 13%]
test/test_page_router.py::test_page_router_profile_history_with_authenticated_user PASSED [ 16%]
test/test_page_router.py::test_page_router_profile_edit_with_authenticated_user PASSED [ 20%]
test/test_page_router.py::test_page_router_profile_edit_password_with_authenticated_user PASSED [ 23%]
test/test_page_router.py::test_page_router_profile_edit_email_with_authenticated_user PASSED [ 26%]
test/test_page_router.py::test_page_router_profile_delete_with_authenticated_user PASSED [ 30%]
test/test_page_router.py::test_page_router_documentation_with_authenticated_user PASSED [ 33%]
test/test_page_router.py::test_page_router_contact_with_authenticated_user PASSED [ 36%]
test/test_page_router.py::test_page_router_forgot_password_with_authenticated_user PASSED [ 40%]
test/test_page_router.py::test_page_router_reset_password_with_authenticated_user PASSED [ 43%]
test/test_page_router.py::test_page_router_error_404_with_authenticated_user PASSED [ 46%]
test/test_page_router.py::test_page_router_profile_unknown_with_authenticated_user PASSED [ 50%]
test/test_page_router.py::test_page_router_home_with_unauthenticated_user PASSED [ 53%]
test/test_page_router.py::test_page_router_launching_with_unauthenticated_user PASSED [ 56%]
test/test_page_router.py::test_page_router_explorer_with_unauthenticated_user PASSED [ 60%]
test/test_page_router.py::test_page_router_profile_history_with_unauthenticated_user PASSED [ 63%]
test/test_page_router.py::test_page_router_profile_edit_with_unauthenticated_user PASSED [ 66%]
test/test_page_router.py::test_page_router_profile_edit_password_with_unauthenticated_user PASSED [ 70%]
test/test_page_router.py::test_page_router_profile_edit_email_with_unauthenticated_user PASSED [ 73%]
test/test_page_router.py::test_page_router_profile_delete_with_unauthenticated_user PASSED [ 76%]
test/test_page_router.py::test_page_router_documentation_with_unauthenticated_user PASSED [ 80%]
test/test_page_router.py::test_page_router_contact_with_unauthenticated_user PASSED [ 83%]
test/test_page_router.py::test_page_router_forgot_password_with_unauthenticated_user PASSED [ 86%]
test/test_page_router.py::test_page_router_reset_password_with_unauthenticated_user PASSED [ 90%]
test/test_page_router.py::test_page_router_error_404_with_unauthenticated_user PASSED [ 93%]
test/test_page_router.py::test_page_router_login_with_unauthenticated_user PASSED [ 96%]
test/test_page_router.py::test_page_router_profile_unknown_with_unauthenticated_user PASSED [100%]

===== 30 passed, 2 warnings in 2.38s =====

```

Figure 18 : Capture d'écran des résultats Pytest pour la fonction de routage

Dans le contexte actuel, avec un nombre gérable de routes à tester, les tests utilisateurs ont été effectués manuellement pour assurer la qualité et la précision de la navigation au sein de l'application. Bien que cette approche ait été efficace, l'évolution future de l'application vers une complexité accrue pourrait bénéficier de l'automatisation des tests. L'utilisation d'outils d'automatisation de tests tels que Selenium permettrait d'effectuer des tests plus étendus et répétitifs sans intervention manuelle, augmentant ainsi l'efficacité des tests et garantissant une couverture plus systématique sur un grand nombre de routes et de scénarios d'utilisation.

X. VEILLE SUR LES VULNERABILITES DE SECURITE

La veille sur les vulnérabilités de sécurité est un aspect crucial du développement et de la maintenance d'une application web. Elle implique une **surveillance continue des nouvelles failles de sécurité rapportées**, ainsi que des stratégies et des pratiques recommandées pour les atténuer.

Durant cette année de formation, j'ai commencé à découvrir tous ces aspects de cybersécurité et j'ai identifié, et régulièrement consultés, plusieurs sites clés sur ce sujet :

- [OWASP](#) : Fondation Open Web Application Security Project, qui publie régulièrement des mises à jour sur les failles de sécurité.
- [CVE](#) : Common Vulnerabilities and Exposures, qui édite une liste de vulnérabilités connues.
- [NIST](#) : National Institute of Standards and Technology, qui offre des informations sur les meilleures pratiques de sécurité.
- [ANSSI](#) : Agence nationale de la sécurité des systèmes d'information, qui fournit des ressources, des conseils et des directives pour renforcer la cybersécurité.
- [CERT-FR](#) : Centre gouvernemental de veille, d'alerte et de réponse aux attaques informatiques, qui procure des alertes, des analyses de menaces et des recommandations pour les professionnels et le grand public en France.

J'ai par exemple **suivi les recommandations de l'ANSSI** sur le format du mot de passe et demandé aux utilisateurs des mots de passe d'au moins 12 caractères, avec au moins une majuscule, une minuscule, un chiffre et un caractère spécial.

J'ai pu aussi me rendre compte que suivre des pages pertinentes sur les réseaux sociaux, s'abonner à des newsletters sur la data, le développement ou la sécurité informatique, et participer à des communautés spécialisées comme des forums ou des canaux Discord peuvent **fournir des alertes en temps réel et un accès à des discussions d'experts** sur les dernières menaces et solutions en matière de cybersécurité.

J'ai listé ci-dessous plusieurs attaques liées à des failles de sécurité communes et les mesures prises ou à prendre dans le cadre du projet EEG Explorer pour s'en protéger :

Attaque par injection SQL (SQLi) : Cette faille survient lorsqu'un attaquant identifie un point d'entrée dans l'application où les données saisies par l'utilisateur sont utilisées pour construire une requête SQL. Ce point d'entrée peut être un formulaire, une URL, ou tout autre input utilisateur. Au lieu de saisir des données normales, l'attaquant injecte une commande SQL malveillante. Par exemple, si l'application attend un nom d'utilisateur, l'attaquant peut saisir « admin'; DROP TABLE users; -- ». Si l'entrée n'est pas correctement filtrée ou échappée, le serveur de base de données peut exécuter la commande malveillante. Dans l'exemple ci-dessus, cela pourrait entraîner la suppression de la table des utilisateurs.

Les conséquences d'une attaque SQLi peuvent être la perte de données, le vol de données, la compromission de l'intégrité des données et ça peut aller jusqu'à la prise de contrôle d'une base de données.

Dans EEG Explorer nous ne concaténons pas de chaîne de caractères pour créer des requêtes SQL mais nous utilisons l'ORM SQLAlchemy qui échappe automatiquement les entrées et utilise des requêtes paramétrées, ce qui réduit considérablement le risque d'injections SQL. De plus, toutes les données entrantes sont vérifiées (voir [ici](#)) et on s'assure qu'elles correspondent au format attendu. Par ailleurs, les permissions de l'utilisateur de la base de données utilisée par l'application sont limitées au strict nécessaire pour empêcher des opérations telles que la suppression de tables ou la modification des droits utilisateurs.

Attaque Cross-Site Scripting (XSS) : XSS est un type de vulnérabilité de sécurité où un attaquant parvient à injecter du code JavaScript ou HTML malveillant dans une page web vue par d'autres utilisateurs. Le but est d'exécuter des scripts dans le navigateur de la victime afin de contourner les mesures de sécurité habituelles pour accéder à des informations sensibles, modifier l'affichage des pages web ou rediriger l'utilisateur vers des sites malveillants.

Il existe plusieurs types d'attaque XSS comme :

- ❖ Reflected XSS : Le script malveillant est reflété par le serveur web, généralement via une URL ou une entrée de formulaire, et exécuté immédiatement dans le navigateur de la victime.
- ❖ Stored XSS : Le script malveillant est stocké sur le serveur web, comme dans une base de données, et exécuté ultérieurement pour tout utilisateur qui accède à la page affectée.
- ❖ DOM-based XSS : Le script malveillant est exécuté à la suite d'une modification du DOM (Document Object Model) de la page web due à une entrée de script côté client, sans nouvelle page chargée du serveur.

Les conséquences d'une attaque XSS peuvent être le vol d'identifiants, le détournement de sessions, le phishing, l'installation de malwares ou de ransomwares, le déni de service (DoS) ou encore la dégradation de la confiance utilisateur. Se protéger contre XSS est donc primordial et implique par exemple la validation et l'échappement systématique des entrées utilisateurs et l'utilisation de Content Security Policy (CSP).

Pour EEG Explorer il est rassurant de savoir que Dash échappe automatiquement le contenu généré par l'utilisateur pour prévenir les attaques XSS lors de l'utilisation de composants Dash standard. Par exemple, si vous utilisez le composant `html.Div`, Dash convertira les caractères spéciaux comme `<` et `>` en entités HTML, ce qui empêche l'exécution de scripts malveillants. De plus, Dash utilise React.js pour le rendu des composants sur le front-end, et React échappe également par défaut tout le contenu entre les balises pour éviter les injections XSS. De plus toutes les entrées côté client et côté serveur passent par une étape de validation, et notamment celles générant du contenu dynamique sur la page. En revanche, Dash n'inclut pas par défaut la mise en place d'une Content Security Policy stricte pour réduire le risque d'exécution de scripts non approuvés, il faudra donc la configurer au niveau du serveur web avant le déploiement.

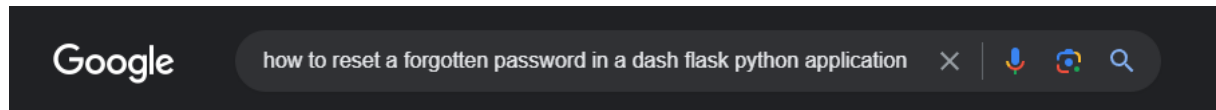
Fuite d'informations : La fuite d'informations dans une application web peut se produire lorsque des détails internes de l'application, tels que des messages d'erreur détaillés, des chemins de fichiers ou des configurations système, sont involontairement exposés aux utilisateurs. Cela peut fournir aux attaquants des indices précieux sur la structure de votre application, la technologie sous-jacente ou même des détails sur les failles de sécurité existantes. Les conséquences peuvent être très graves comme la perte de la confidentialité des données, l'atteinte à la réputation et la perte de confiance des utilisateurs, risques juridiques et de conformité, exposition à d'autres attaques (phishing, usurpation d'identité, ...) ou encore la perte de propriété intellectuelle.

Pour s'en prémunir, il est important de faire très attention à la personnalisation des messages d'erreur. Avec Flask il est possible d'utiliser l'objet `Flask.errorhandler` pour personnaliser les réponses d'erreur et masquer les traces de pile et les messages de débogage qui pourraient être révélateurs. D'une manière générale la bonne configuration de l'environnement en production et la désactivation du débogage sont importantes. Il faut aussi mettre en place une stratégie de journalisation qui capture les erreurs sans stocker ou afficher d'informations sensibles et utiliser des outils de surveillance pour détecter et alerter sur les comportements anormaux qui pourraient indiquer une fuite d'informations. Il est bien sûr très important de stocker les clés API, les mots de passe et les secrets dans des variables d'environnement ou dans un gestionnaire de secrets sécurisé, jamais dans le code source ou dans des fichiers de configuration versionnés (utilisation du `.gitignore`).

D'une manière générale, il est essentiel de revoir régulièrement le code et de **maintenir à jour toutes les dépendances de l'application** pour s'assurer d'utiliser les versions les plus sécurisées.

XI. RECHERCHE DE SOLUTION EN SITUATION DE TRAVAIL

Quand je me suis penchée sur la fonctionnalité de réinitialisation d'un mot de passe oublié je ne savais pas comment faire. J'ai donc effectué des recherches sur internet :



Parmi les premières réponses, j'ai trouvé ce tutoriel : [Flask mail Reset Password with Token](#) qui m'a permis de découvrir la bibliothèque python « [It's Dangerous](#) » et j'ai donc consulté la documentation pour savoir si 1) ça répondait bien à mon besoin et 2) comment j'allais pouvoir l'utiliser.



Sometimes you want to send some data to untrusted environments, then get it back later. To do this safely, the data must be signed to detect changes.

Given a key only you know, you can cryptographically sign your data and hand it over to someone else. When you get the data back you can ensure that nobody tampered with it.

The receiver can see the data, but they can not modify it unless they also have your key. So if you keep the key secret and complex, you will be fine.

Figure 19 : Page d'accueil de la documentation de It's Dangerous

Et en effet, en utilisant le tutoriel et la documentation de « It's Dangerous » j'ai pu ajouter les deux méthodes suivantes dans ma classe User pour générer un token et pour pouvoir le vérifier :

```
def get_reset_token(self, expires_sec=900):
    s=Serializer(api_flask.secret_key, expires_sec)
    token = s.dumps(
        {'user_id': self.id},
        salt=os.environ.get('salt')
    ).decode('utf-8')
    return token
```

La méthode **get_reset_token** génère un token de réinitialisation de mot de passe. Elle utilise un objet Serializer (issu de la bibliothèque It's Dangerous) avec la clé secrète de l'API (`api_flask.secret_key`) pour créer un token sécurisé. Le token contient l'identifiant de l'utilisateur (`user_id`) et est encodé en UTF-8. Le paramètre **expires_sec** définit le temps d'expiration du token (par défaut, 900 secondes, soit 15 minutes). Un **salt** (stocké dans les variables d'environnement) est aussi utilisé pour renforcer la sécurité en rendant le hash du token unique.

```

@staticmethod
def verify_reset_token(token, expires_sec=900):
    s=Serializer(api_flask.secret_key)
    try:
        user_id = s.loads(
            token,
            expires_sec,
            salt= os.environ.get('salt'),
        )['user_id']
    except:
        return None
    return User.query.get(user_id)

```

La méthode statique **verify_reset_token** vérifie la validité d'un token de réinitialisation. Elle décode le token en utilisant la même clé secrète et le même salt. Si le token est valide et n'a pas expiré, elle extrait l'**user_id** et renvoie l'instance de l'utilisateur correspondante. En cas d'échec (token invalide ou expiré), elle renvoie None.

Grace à cela, **la fonctionnalité de gestion des mots de passe oubliés** que j'ai développée suit un processus clair et sécurisé. Lorsqu'un utilisateur oublie son mot de passe, il peut cliquer sur un lien dédié situé sous le formulaire de connexion. Ce lien le redirige vers une page où il doit saisir son adresse email. Si celle-ci est associée à un compte utilisateur existant, un courriel contenant un lien de réinitialisation est envoyé à cette adresse. Ce lien conduit à l'URL `/reset/password/{token}`. Sur cette page, l'utilisateur est invité à entrer son nouveau mot de passe deux fois pour confirmation. Après avoir cliqué sur le bouton de réinitialisation, le système vérifie la validité du token. Si le token est valide et que le nouveau mot de passe répond aux critères requis, les informations du compte utilisateur sont mises à jour dans la base de données. **Ce processus garantit une récupération sécurisée du compte tout en restant simple et convivial pour l'utilisateur.**

XII. CONCLUSION

Conduire le projet EEG Explorer a été **une aventure révélatrice et enrichissante**. Plus qu'une simple combinaison de compétences techniques, ce projet a été le témoin de la progression que j'ai réalisée depuis le début de ma reconversion il y a deux ans. J'ai été particulièrement fière de **ma capacité à m'adapter et à maîtriser des technologies nouvelles**, allant au-delà de ce qui a été couvert en formation et explorant des domaines inconnus même pour mes collègues.

Le projet a atteint ses objectifs, avec **la création d'un prototype fonctionnel permettant de consulter en langage naturel la base de données Spectre**. L'anticipation de la mise en service de la base de données réelle de Spectre et l'opportunité de tester et d'améliorer les performances du modèle d'IA sont des perspectives excitantes. Je suis également rassurée de savoir que **l'infrastructure de l'application est déjà prête**, répondant aux critères de sécurité, de design et d'accessibilité définis au début du projet.

En allant au-delà du domaine de la data, ce projet m'a permis de **développer une application** de A à Z, d'appréhender **l'architecture multicouche**, d'apprécier l'importance de la **méthodologie de projet**, et de m'initier à des outils comme Figma et à des frameworks tels que Bootstrap, Flask et Langchain.

Ce parcours a renforcé ma passion pour **la résolution de problèmes, l'implémentation de solutions innovantes, et la conception de fonctionnalités centrées sur l'utilisateur**. La curiosité que j'ai pour l'évolution de l'IA, et son impact potentiel sur le secteur de la santé et des neurosciences, ne fait que croître.

Ces deux années ont **confirmé mon choix de reconversion professionnelle**. Aujourd'hui, je suis extrêmement enthousiaste à l'idée de **poursuivre mon aventure chez Spectre Biotech** en tant que Data Scientist polyvalente. Entourée d'une équipe bienveillante et passionnée, je me sens **prête à relever les défis à venir**, consolidant ainsi ma transition vers ce domaine fascinant.

XIII. ANNEXES

1. ANNEXE 1 : EXEMPLES D'ELEMENTS ET DE REQUETES

Exemples d'éléments à relever et de requêtes fournis par le CEO de Spectre Biotech au moment du lancement du projet :

Eléments basiques à relever :

- Genre
- Date de naissance
- NSC (diplôme)
- Latéralité

Eléments cliniques à relever :

- Arbre questionnaire clinique

Eléments EEG à relever :

- Fréquence du pic alpha
- La puissance spectrale (mesurée sur toutes les électrodes, pour toutes les fréquences, en absolu et relatif)
- La cohérence de phase (mesurée sur toutes les paires d'électrodes)

Eléments statistiques recherchés :

- Moyenne
- Variance
- Ecart-type
- Médiane

Requêtes :

- Je veux connaître pour la tranche d'âge 20-25 qui sont **autistes** la **puissance spectrale Alpha moyenne** (avec variance, écart-type, médiane) pour les électrodes O1 Oz O2 P3 Pz et P4
- Je veux connaître pour les droitiers de 50 ans hommes qui ont eu **un AVC de l'artère sylvienne droite** la cohérence de phase moyenne (avec variance, écart-type, médiane) entre **les électrodes FP1 et FP2**
- Je veux connaître pour la tranche d'âge 60-100 qui sont **sains** (pas de structures de santé **et rien dans le questionnaire clinique**) la **fréquence du pic alpha moyen** (avec variance, écart-type, médiane)

2. ANNEXE 2 : DEROULE TEMPOREL DU PROJET EEG EXPLORER

Mois	1	2	3	4	5	6	7	8	9	10	11	12
Lancement du projet												
Analyse des besoins												
Définition des cas d'usage												
Création des maquettes												
Choix des technologies												
Recherches sur l'IA												
Front-end												
Tests et réglages IA												
Bases de données												
API + Back-end												
Réflexion déploiement												
Tests												
Veille technologique sur la sécurité												

3. ANNEXE 3 : PREMIERS WIREFRAMES D'EEG EXPLORER



4. ANNEXE 4 : CHARTE GRAPHIQUE D'EEG EXPLORER

❖ LA DEMARCHE

La demande des décideurs était de créer une identité propre à EEG Explorer en gardant la couleur principale du logo (Penn Blue ■) et la typographie du site internet (Inter). J'ai donc commencé par créer une palette de couleur avec le site coolers.co avec des nuances de bleu allant du Penn Blue au blanc. A partir de cette palette j'ai demandé à DALL-E (l'intelligence artificielle de création d'images d'OpenAI) de créer le logo qui servira aussi de favicon pour le site. J'ai également modifié la colorimétrie d'une image d'archive pour créer l'en-tête de l'application. En complément de la typographie Inter de Google fonts, utilisée pour tous les textes de l'application, j'ai choisi la typographie Nort d'Adobe fonts pour les titres et sous titres, une typographie ressemblante à celle du logo de Spectre Biotech (voir en annexes).

Au fil de la construction de la maquette, quand des images étaient nécessaires, elles ont également été créées par IA sur la base de la palette de couleur.

❖ LA PALETTE DE COULEUR



❖ LE LOGO



Format carré 1/1

Prompt : create a logo representing a brain containing an EEG wave seen through a magnifying glass (the format as to be 1:1) the main color has to be penn blue (hex=1A1950) and the other color that you can use are in the joined image (color palette).

❖ L'EN-TETE

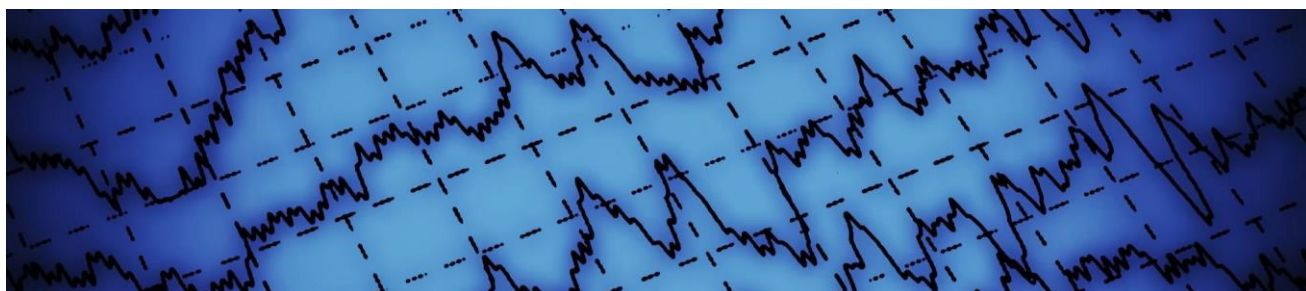


Image d'archive libre de droit avec colorimétrie modifiée.

❖ IMAGES GENEREES PAR IA

Explorer

Consulter l'historique

Se déconnecter

Modifier son profil



Page introuvable (erreur 404)



❖ TYPOGRAPHIES

Nort Ultra (adobe font)

abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789;:/!?

Inter regular (google font)

abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789;:/!?

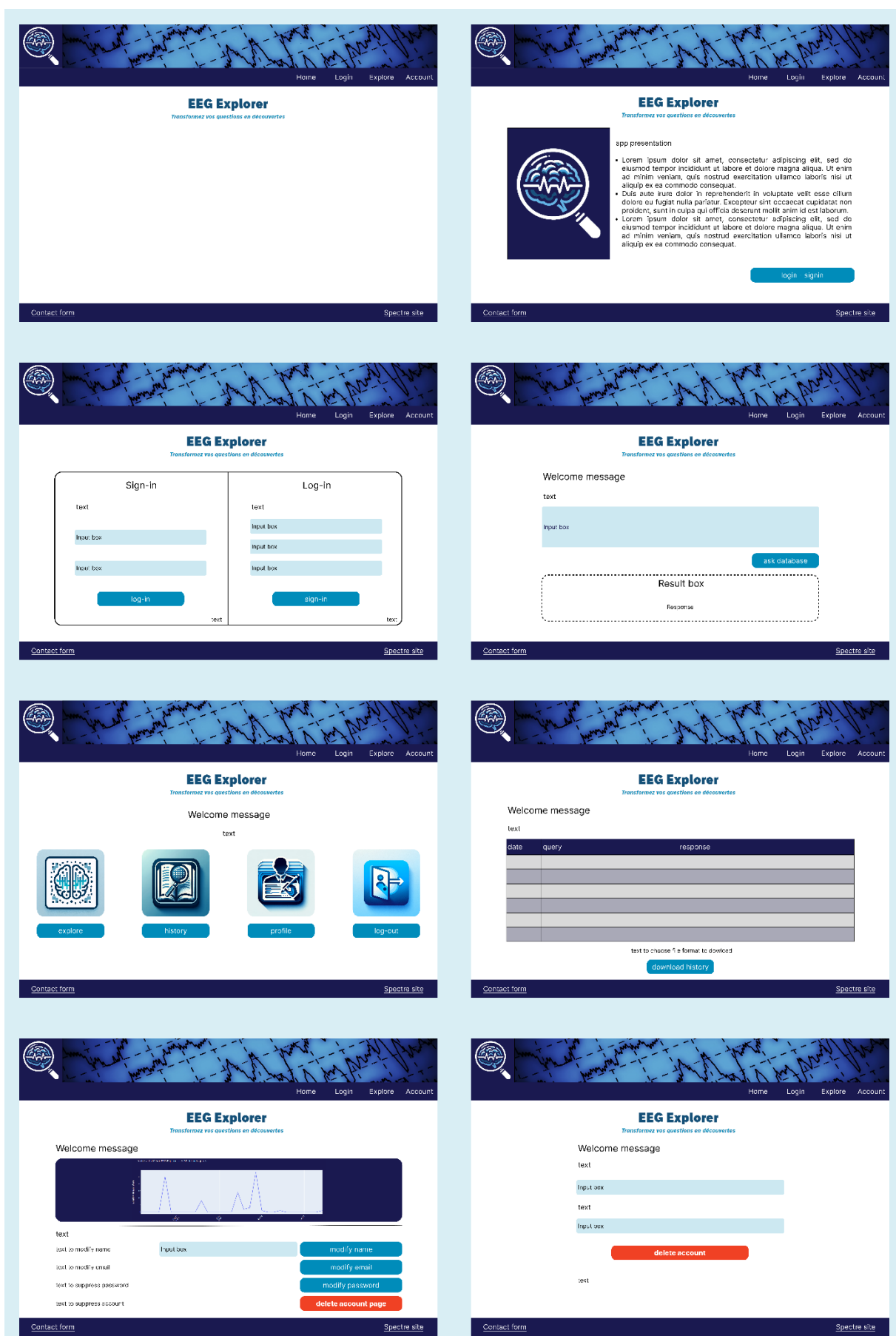
❖ ANNEXES

Logos de l'entreprise Spectre Biotech :



SPECTRE
BIOTECH.

5. ANNEXE 5 : MAQUETTES DE L'APPLICATION EEG EXPLORER



```
""" Configuration file for the SQL agent. """

import os

from dotenv import load_dotenv
from langchain.agents import initialize_agent
from langchain.agents.agent_toolkits import SQLDatabaseToolkit
from langchain.agents.agent_types import AgentType
from langchain.callbacks import get_openai_callback
from langchain.chat_models import ChatOpenAI
from langchain.utilities import SQLDatabase

load_dotenv()

url_database = os.environ.get('DATABASE_URL_SPECTRE')

database = SQLDatabase.from_uri(url_database)

model_name = 'gpt-4-turbo-preview'

llm_model = ChatOpenAI(model_name=model_name)

toolkit = SQLDatabaseToolkit(db=database, llm=llm_model)

tools = toolkit.get_tools()

agent_type = AgentType.ZERO_SHOT_REACT_DESCRIPTION

PREFIX = ...

SUFFIX = ...

prompt_template = {
    'prefix': PREFIX,
    'suffix': SUFFIX
}
```

Les consignes exactes données au modèle ne sont pas divulguée par demande de discrétion de l'entreprise Spectre Biotech.

7. ANNEXE 7 : EXEMPLE DU FONCTIONNEMENT DE L'AGENT SQL

Question : « Quelle est la valeur moyenne du pic alpha en F3 chez les personnes souffrant d'autisme »

> Entering new AgentExecutor chain...

1 - Action: `sql_db_list_tables`

1 - Action Input:

1 - Observation: `alpha_peak, diagnostics, ercoh, ersp, evaluations, power_spectral_density, schoollevels, users`

1 - Thought: The table that seems most relevant to the question is likely "alpha_peak," as it directly references the alpha peak which is related to the brainwave data mentioned in the question. However, I may also need to look at "diagnostics" if that table contains information regarding conditions such as autism. I should check the schema for these tables to understand their structures and the fields they contain.

2 - Action: `sql_db_schema`

2 - Action Input: `alpha_peak, diagnostics`

2 - Observation:

```
CREATE TABLE alpha_peak (  
    value_type VARCHAR NOT NULL,  
    electrode VARCHAR NOT NULL,  
    id BIGINT GENERATED BY DEFAULT AS IDENTITY,  
    evaluation_id BIGINT NOT NULL,  
    value DOUBLE PRECISION NOT NULL,  
    CONSTRAINT alpha_peak_pkey PRIMARY KEY (id),  
    CONSTRAINT alpha_peak_evaluation_id_fkey FOREIGN KEY(evaluation_id) REFERENCES  
evaluations (id)  
)
```

/*

3 rows from alpha_peak table:

value_type	electrode	id	evaluation_id	value
peak Fp1	1	201		13.469031940419349
frequency	Fp1	2	201	7.75
peak Fp2	3	201		13.36482891834609

*/

```
CREATE TABLE diagnostics (  
    id BIGINT GENERATED BY DEFAULT AS IDENTITY,  
    disease VARCHAR NOT NULL,  
    CONSTRAINT diagnostics_pkey PRIMARY KEY (id)  
)
```

/*

3 rows from diagnostics table:

```
id    disease
1     Autisme
2     Dépression
3     Epilepsie
*/
```

2 - Thought: To answer the question, I need to join the "alpha_peak" and "diagnostics" tables using the "evaluation_id" field from "alpha_peak" and the "id" field from "diagnostics" to filter for individuals with autism. Specifically, I'm interested in the average value ("value") of the alpha peak in electrode F3 for those individuals.

I have to ensure that I'm selecting the average value where the "value_type" is 'peak', the "electrode" is 'F3', and the "disease" is 'Autisme'. I should create a query that reflects this.

3 - Action: `sql_db_query_checker`

3 - Action Input: `SELECT AVG(ap.value) AS average_alpha_peak
FROM alpha_peak ap
INNER JOIN evaluations e ON ap.evaluation_id = e.id
INNER JOIN diagnostics d ON e.diagnostic_id = d.id
WHERE ap.value_type = 'peak'
AND ap.electrode = 'F3'
AND d.disease = 'Autisme';`

3 - Observation: ````sql
SELECT AVG(ap.value) AS average_alpha_peak
FROM alpha_peak ap
INNER JOIN evaluations e ON ap.evaluation_id = e.id
INNER JOIN diagnostics d ON e.diagnostic_id = d.id
WHERE ap.value_type = 'peak'
AND ap.electrode = 'F3'
AND d.disease = 'Autisme';
````

3 - Thought: There seems to be an error with the query as it was formatted in the checker. I need to correct the query to make sure it is syntactically correct and that it uses the correct table relationships. I will assume it was an error and try to correct the query. I realized that I made a mistake assuming that there is a 'diagnostic\_id' in the 'evaluations' table when such a column was not confirmed in the schema. I should verify the schema for the 'evaluations' table to correct the query accordingly.

4 - Action: `sql_db_schema`

4 - Action Input: `evaluations`

4 - Observation:

```
CREATE TABLE evaluations (
 id BIGINT GENERATED BY DEFAULT AS IDENTITY,
 user_id BIGINT NOT NULL,
 date TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
 CONSTRAINT evaluations_pkey PRIMARY KEY (id),
 CONSTRAINT evaluations_user_id_fkey FOREIGN KEY(user_id) REFERENCES users (id)
)
```

/\*

3 rows from evaluations table:

| id  | user_id | date                      |
|-----|---------|---------------------------|
| 201 | 66      | 2022-01-10 00:00:00+01:00 |
| 202 | 80      | 2023-12-04 00:00:00+01:00 |
| 203 | 85      | 2022-07-02 00:00:00+02:00 |

\*/

4 - Thought: The "evaluations" table does not directly reference the "diagnostics" table. To properly join them, there must be a relationship between the users diagnosed with autism and their evaluations. Since no such direct relationship is visible in the provided schemas, I will need to check if there is an intermediary table or column that relates "users" to "diagnostics".

5 - Action: `sql_db_schema`

5 - Action Input: `users`

5 - Observation:

```
CREATE TABLE users (
 id BIGINT GENERATED BY DEFAULT AS IDENTITY,
 gender VARCHAR NOT NULL,
 birthdate TIMESTAMP WITHOUT TIME ZONE NOT NULL,
 schoollevel_id BIGINT NOT NULL,
 laterality VARCHAR NOT NULL,
 has_diagnostic BOOLEAN NOT NULL,
 diagnostic_id BIGINT,
 CONSTRAINT users_pkey PRIMARY KEY (id),
 CONSTRAINT users_diagnostic_id_fkey FOREIGN KEY(diagnostic_id) REFERENCES
diagnostics (id),
 CONSTRAINT users_schoollevel_id_fkey FOREIGN KEY(schoollevel_id) REFERENCES
schoollevels (id)
)
```

/\*

3 rows from users table:

| id | gender | birthdate  | schoollevel_id | laterality | has_diagnostic | diagnostic_id |
|----|--------|------------|----------------|------------|----------------|---------------|
| 5  | M      | 1981-08-31 | 4              | R          | True           | 7             |
| 6  | F      | 1979-02-20 | 1              | R          | True           | 5             |
| 7  | M      | 2002-08-30 | 1              | L          | True           | 8             |

\*/

5 - Thought: Now I see that the "users" table contains a "diagnostic\_id" column that references the "diagnostics" table. I can use this to join the "users" with the "diagnostics" and then join the "evaluations" table with the "users" table based on "user\_id". This will allow me to filter for users with a diagnostic of autism and then find the average alpha peak value at electrode F3 for their evaluations. Now I can correct the query and check it again.

6 - Action: `sql_db_query_checker`

6 - Action Input: `SELECT AVG(ap.value) AS average_alpha_peak`

`FROM alpha_peak ap`

`INNER JOIN evaluations e ON ap.evaluation_id = e.id`

`INNER JOIN users u ON e.user_id = u.id`

`INNER JOIN diagnostics d ON u.diagnostic_id = d.id`

`WHERE ap.value_type = 'peak' AND ap.electrode = 'F3' AND d.disease = 'Autisme';`

6 - Observation: **Here is the provided query, which appears to be free from the common mistakes listed:**

````sql`

`SELECT AVG(ap.value) AS average_alpha_peak`

`FROM alpha_peak ap`

`INNER JOIN evaluations e ON ap.evaluation_id = e.id`

`INNER JOIN users u ON e.user_id = u.id`

`INNER JOIN diagnostics d ON u.diagnostic_id = d.id`

`WHERE ap.value_type = 'peak'`

`AND ap.electrode = 'F3'`

`AND d.disease = 'Autisme';`

`````

6 - Thought: **The query has been checked and is now ready to be executed. I can proceed to get the results.**

7 - Action: `sql_db_query`

7 - Action Input: `SELECT AVG(ap.value) AS average_alpha_peak`

`FROM alpha_peak ap`

`INNER JOIN evaluations e ON ap.evaluation_id = e.id`

`INNER JOIN users u ON e.user_id = u.id`

`INNER JOIN diagnostics d ON u.diagnostic_id = d.id`

`WHERE ap.value_type = 'peak' AND ap.electrode = 'F3' AND d.disease = 'Autisme';`

7 - Observation: **`[(-21.15102194338327,)]`**

7 - Thought: **I now know the final answer**

**> Final Answer : La valeur moyenne du pic alpha en F3 chez les personnes souffrant d'autisme est de -21.15102194338327.**